



Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie

Wydział Informatyki

PRACA DYPLOMOWA

Tsetlin Machines Execution Engine for Embedded Devices

Silnik wykonawczy Maszyn Tsetlina dla urządzeń o ograniczonych zasobach

Autorzy:	Robert Mesek, Igor Sikora
Kierunek:	Informatyka
Opiekunowie pracy:	dr hab. inż. Bartłomiej Śnieżyński, prof. AGH dr hab. inż. Tomasz Szydło, prof. AGH

Kraków, 2026

In the preparation of this engineering thesis, AI-powered language tools were utilized to enhance the clarity, coherence, and overall quality of the written text. These tools assisted in tasks such as grammar correction, style refinement, and suggested phrasing. However, the content, ideas, research findings, and conclusions presented herein are solely the author's own. Any errors or omissions remain the responsibility of the author.

Streszczenie

Niniejsza praca przedstawia projekt i implementację silnika wykonawczego Maszyny Tsetlina (TM) w języku programowania C, opracowanego z myślą o wymaganiach urządzeń wbudowanych. Głównym wkładem jest stworzenie od podstaw wariantów Coalesced TM oraz Sparse TM. Rozwiązanie to umożliwia zarówno inferencję, jak i uczenie bezpośrednio na urządzeniu, przy wykorzystaniu wydajnego formatu serializacji FlatBuffers. Ewaluacja na zbiorach danych Noisy-XOR oraz Digits demonstruje potencjał TM w środowiskach o ograniczonych zasobach. Wyniki podkreślają znaczenie doboru hiperparametrów oraz wyzwania związane z uczeniem przyrostowym, w szczególności potrzebę zachowania równowagi między retencją wiedzy a przyswajaniem nowych wzorców. Opracowane oprogramowanie wraz z narzędziami wspierającymi zostało udostępnione jako projekt otwartoźródłowy, aby ułatwić stosowanie TM w rzeczywistych scenariuszach.

Abstract

This work presents the design and implementation of a Tsetlin Machine (TM) execution engine in the C programming language, developed to meet the requirements of embedded devices. The main contribution is the development of Coalesced TM and Sparse TM variants from scratch. This solution enables both inference and on-device training while utilizing the efficient FlatBuffers serialization format. Evaluations on the Noisy-XOR and Digits datasets demonstrate the potential of TMs in resource-constrained environments. The results highlight the importance of hyperparameter selection and the challenges associated with incremental learning, specifically the need to balance knowledge retention with the acquisition of new patterns. The developed software and supporting tools have been released as an open-source project to simplify the application of TMs in real-world scenarios.

Contents

List of Figures	xi
List of Tables	xiii
List of Listings	xv
1. Project Goals and Vision	1
1.1. Competitive Solutions	2
1.2. Power Usage in Machine Learning	2
1.3. Thesis Structure	2
2. Theoretical Background	5
2.1. Tsetlin Machine Overview	5
2.2. Tsetlin Automaton	6
2.3. Architecture and Problem Representation	7
2.4. Learning Algorithm	10
2.5. Tsetlin Machine Architecture Variants	14
2.6. Comparison with Neural Networks	17
2.7. Summary	18
3. Functional Scope	19
3.1. Expected User Groups	19
3.2. Functional Requirements	19
3.3. Nonfunctional Requirements	20
4. Selected Realization Aspects	23
4.1. System Architecture and Data Workflow	23
4.2. Design	25
4.2.1. Programming Language	25
4.2.2. Execution Engine	25
4.2.3. Model Serialization Library	25
4.3. TM implementation	26
4.3.1. Tsetlin Automaton	26
4.3.2. Coalesced TM	26

4.3.3. Sparse TM	30
4.3.4. Stateless TM	31
4.4. Optimizations	32
4.4.1. Pseudo Random Number Generation	32
4.5. Summary	35
5. Work Organization	37
5.1. Team Members and Their Roles	37
5.2. Used Tools	38
6. Technical Documentation	39
6.1. Case Study	39
6.2. API	45
6.2.1. TsetlinMachine	45
6.2.2. SparseTsetlinMachine	46
6.2.3. StatelessTsetlinMachine	46
6.3. Build	46
6.3.1. Build Requirements	46
6.3.2. Build Options	47
6.4. Installation	47
6.4.1. Install with Python	47
6.5. Testing and Examples	47
7. Evaluation	49
7.1. Noisy-XOR Dataset	51
7.2. Digits Dataset	54
7.3. Fashion-MNIST Dataset	58
7.4. Incremental Noisy-XOR Learning	60
7.5. Embedded Noisy-XOR Benchmarks	62
7.6. Summary	64
8. Conclusions	65
8.1. Achieved Results	65
8.2. Further Work	65
8.2.1. Cross Compilation	65
8.2.2. Coalesced TM and the Linear Layer	66
8.2.3. Integer Only Implementation of TM	66
Bibliography	67

List of Figures

2.1. The TM inference structure	6
2.2. A Tsetlin Automaton for two-action environment	7
2.3. Two Tsetlin Automata teams	9
2.4. Sticky memory for IMDB example	10
2.5. Type Ia feedback for IMDB example	11
2.6. Type Ib feedback for IMDB example	12
2.7. Type II feedback for IMDB example	12
2.8. TM XOR-gate training	14
2.9. The Coalesced TM structure	15
2.10. The Sparse TM Active Literals	16
2.11. Decision boundary visualization	17
4.1. System architecture and data workflow	24
4.2. Clause data layout of Coalesced TM	29
4.3. Clause data layout of Sparse TM	31
4.4. Clause data layout of Sparse TM (Stateless)	32
7.1. Five samples from the Digits dataset	54
7.2. Five samples from the Fashion-MNIST dataset	58
7.3. Train and test accuracy per batch on the Noisy-XOR	61

List of Tables

4.1.	Comparison of model serialization libraries	26
7.1.	Tsetlin Machine default parameters	50
7.2.	Noisy-XOR hyperparameters	52
7.3.	Noisy-XOR benchmark results	53
7.4.	Digits hyperparameters	56
7.5.	Digits dataset benchmark results	57
7.6.	Fashion-MNIST hyperparameters	59
7.7.	Fashion-MNIST benchmark results	59
7.8.	Noisy-XOR performance on Raspberry Pi Pico	63

List of Listings

4.1. Coalesced TM FlatBuffers serialization format	30
4.2. <i>green_tsetlin</i> [1] implementation of <code>wyhash64</code>	33
4.3. <i>tsetlin_machine_c</i> implementation of <code>xorshift32</code>	34
6.1. Example Python data preparation steps	40
6.2. Example Python model fitting	41
6.3. Example Python model evaluation	41
6.4. Example Python model export to FlatBuffers format	41
6.5. Example C model import	42
6.6. Example C data loading	42
6.7. Example C model fitting	43
6.8. Example C model evaluation	43
6.9. Example C deallocation	43
6.10. Example C model initialization	44
6.11. Project build commands	46
6.12. Library installation commands	47
6.13. Python environment synchronization	47
6.14. Running accuracy tests	48
6.15. Running MNIST examples	48

1. Project Goals and Vision

Machine learning architectures are becoming a popular topic of research as they continue to show promise in various use cases. Recent years have shown rapid developments in this field, aimed especially at improving model accuracy and scaling. Not all applications can benefit from these advancements, however, as they typically suffer from issues such as low model explainability, high computation power requirements, and high energy usage. This project explores how an implementation of the Tsetlin Machine (TM) machine learning architecture could bridge these gaps and allow for an efficient, lightweight execution engine that can train and infer even on edge devices. To allow transferring a model to and from the edge device, a lightweight model serialization format was developed.

The Tsetlin Machine is a logic-based, interpretable machine learning model that builds propositional formulas (conjunctive clauses of literals) utilizing large collectives of simple finite-state Tsetlin Automata which learn via reward/penalty feedback. It offers a transparent, lighter alternative to neural networks and is notable for simple, hardware-friendly primitives that can yield competitive accuracy while making learned rules easy to inspect.

Due to these properties, it is the most suitable choice for resource-limited environments. To utilize this technology in this scenario two things are needed: a Tsetlin Machine execution engine capable of operating with limited resources, together with a supported, lightweight model serialization format.

The Tsetlin Machine execution engine, to be able to run on edge devices, is limited in a number of ways. Firstly, it has to be implemented in a low-level programming language – C was chosen for that purpose. Secondly, the choice of libraries that can be utilized is severely limited. And finally, the implementation itself has to be optimized to avoid unnecessarily using too much resources.

The Tsetlin Machine model serialization format is another major topic of this project. Due to the many limitations, popular ML model serialization formats like ONNX or Protocol Buffers could not be used. Either the library itself is too big and resource-intensive, or the resulting model file is too large or cannot be memory-mapped. In this case, the only fitting option was found to be a minimalist C implementation of FlatBuffers.

1.1. Competitive Solutions

According to the authors' knowledge, *green_tsetlin* [1] is the only production-ready implementation of the TM. It is primarily a Python library with a C++ backend. A trained model can then be exported into a C header file and used for inference. This approach was found lacking in a number of ways;

1. the exported model cannot be interpreted back to its previous form to perform modifications on it
2. in particular, it makes training and fine-tuning the model impossible
3. the model serialization is saved in a text format, which in this case allows inference without a separate interpreter, but otherwise means wasting a lot of storage space

There are also numerous projects with far less mature implementations that were taken into consideration that did not classify for this reason. [2, 3, 4]

1.2. Power Usage in Machine Learning

Recent advances in machine learning have been accompanied by a significant increase in computational and energy demands. Large-scale models and complex training procedures often require substantial processing power, specialized hardware, and extended execution times, which directly translate into higher power consumption – resulting in high costs, energy grid strain and environmental damages.

While such approaches are justified for problems that require high model capacity or large-scale data processing, not all tasks require the use of computationally intensive methods. For certain problem classes, simpler models or lightweight algorithms can achieve comparable performance with considerably lower energy and resource requirements.

Recognizing the trade-off between model complexity and power usage is therefore essential. Selecting an appropriately scaled solution can reduce energy consumption, improve efficiency, and practical deployability without compromising the intended functionality.

1.3. Thesis Structure

This thesis is organized into several chapters that collectively describe the theoretical foundations, design decisions, implementation, and evaluation of the proposed work.

Chapter 2 – the theoretical background relevant to this thesis. It introduces the fundamental concepts, models, and prior work required to understand the problem domain and motivates the chosen approach.

Chapter 3 – the functional scope of the system. It outlines the objectives, functional and non-functional requirements.

Chapter 4 – selected realization aspects. This chapter focuses on key design and implementation decisions, including important architectural choices and algorithms.

Chapter 5 – the organization of the work. It explains the task division and tooling, providing insight into how the project was structured and executed.

Chapter 6 – the user documentation, detailing how the Tsetlin Machine execution engine is configured, operated, and extended. This chapter is intended to support practical use of the developed solution.

Chapter 7 – the experimental setup and results. It describes the evaluation methodology, test scenarios, and measured outcomes used to assess performance and behavior.

Chapter 8 – the results of the work. It summarizes the contributions of the thesis and outlines potential directions for future research and development.

2. Theoretical Background

This chapter introduces the Tsetlin Machine, a method for pattern recognition based on propositional logic and game theory. We briefly examine the ideas developed in the original research [5, 6] that were necessary for our work, explaining how the model is designed from the ground up. Finally, we discuss scenarios where this approach has the potential to be a viable alternative to neural networks. For a more extensive introduction to Tsetlin Machines, refer to [7, 8].

2.1. Tsetlin Machine Overview

The emergence of the TM is rooted in the development of a lesser-known, yet fundamental and versatile learning mechanism called the Tsetlin Automaton, developed by M.L. Tsetlin in the early 1960s [5, 9]. Further research on this topic proposes a novel technique for complex large scale classification constructed using Tsetlin Automata [5].

Pattern recognition The most important distinction of the TM lies in how it addresses pattern recognition problems. While the accuracy of a machine learning technique is related to its capability to represent patterns, the TM decomposes complex problems into self-contained sub-patterns [5]. Instead of using the complex arithmetic found in neural networks, this method utilizes Boolean algebra, which is the basis of how digital computers operate. As a result, the machine captures the structure of the data by expressing patterns as conjunctive clauses in propositional logic.

Logic-based approach The TM processes input features as bits to construct propositional logic expressed as “If-Then” rules, which offers both computational simplicity and interpretability. In short, given an input vector of propositional variables $X = (x_1, \dots, x_n) \in \{0, 1\}^n$, we add their negated counterparts $\bar{x}_k = 1 - x_k$ to build a set of literals $L = \{l_1, \dots, l_{2n}\}$. From these literals, we construct conjunctive clauses that determine the output vector $Y = (y_1, \dots, y_m) \in \{0, 1\}^m$. For example, clause $C(X) = x_1 \wedge x_2 = x_1 x_2$ consists of literals $\{x_1, x_2\}$ and outputs 1 if and only if $x_1 = x_2 = 1$ [5]. The simple inference structure of the TM can be seen in Figure 2.1.

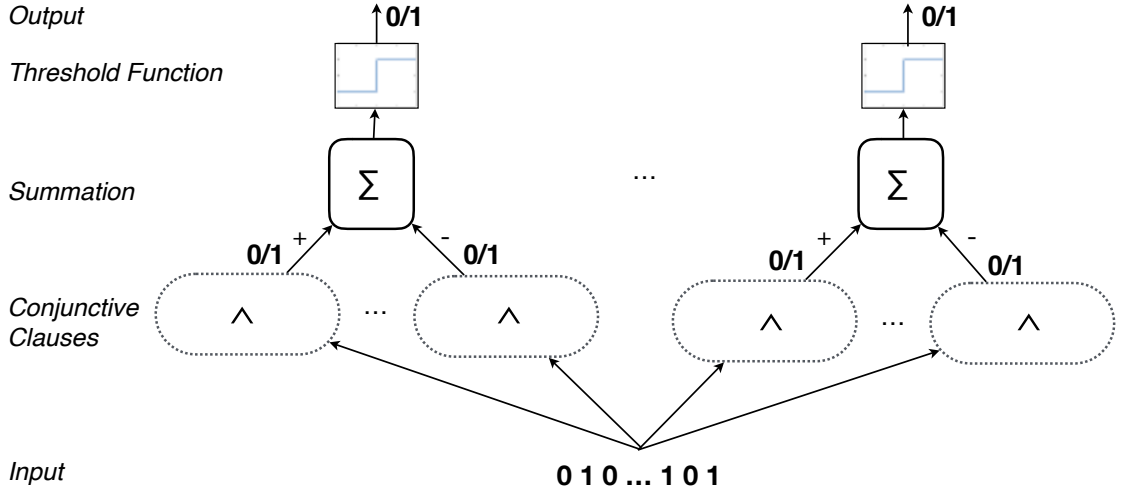


Figure 2.1.: The TM inference structure (source: [5])

2.2. Tsetlin Automaton

The basic building block of a TM is the Tsetlin Automaton (Figure 2.2), recognized as one of the first solutions to the multi-armed bandit problem [5]. It is capable of learning the optimal action when operating in unknown stochastic environments and is considered the first Learning Automaton [10].

Decision making The Tsetlin Automaton performs actions $\alpha_z, z \in \{1, 2\}$ sequentially in an environment. Each action α_z triggers a reward or a penalty. The action is rewarded with probability p_z , otherwise it is penalized. The reward probabilities are unknown to the automaton and may change over time. Under these conditions, the goal is to identify the action with the highest probability of reward using as few attempts as possible [5].

State-based memory The Tsetlin Automaton operates as a finite-state machine with $2N$ states, as illustrated in Figure 2.2. The current state determines the action: action α_1 is performed in states 1 to N , while action α_2 is performed in states $N + 1$ to $2N$. State transitions control how the automaton learns. One set of state transitions is activated upon receiving a reward (solid lines in the figure), and another set of state transitions is activated upon receiving a penalty (dotted lines in the figure). This mechanism is designed to reinforce successful (reward producing) actions [5].

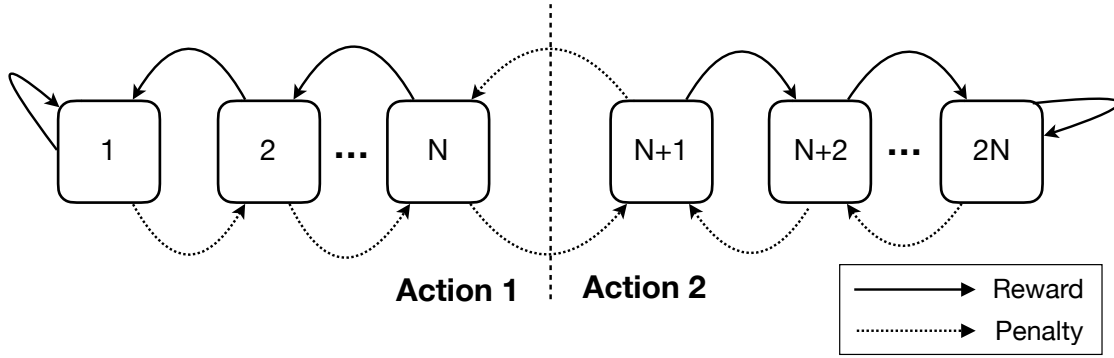


Figure 2.2.: A Tsetlin Automaton for two-action environment (source: [5])

2.3. Architecture and Problem Representation

With the Tsetlin Automaton introduced as the main learning mechanism, the TM architecture arranges these automata into teams responsible for composing clauses. These teams collaborate to solve complex problems by optimizing pattern recognition accuracy. The generated clauses function as rules that vote for or against a specific class output while providing transparent logic to reach a final decision [5]. However, this approach requires specific input, where the data must be processed to produce features compatible with logic-based reasoning.

Binarization of data Since the TM operates using propositional logic, the input data is required to be represented as Boolean features. Nevertheless, this constraint can be mitigated with a simple pre-processing step that transforms the input into a suitable binary format. For categorical data, the universal one-hot encoding approach will suffice. For example, in the digit recognition problem with 8×8 images (64 elements) and 16 possible brightness levels, the sample can be flattened and expanded to a binary vector. The handling of continuous features presents a different challenge, commonly addressed by segmenting data into predefined intervals. A continuous variable, such as temperature, can be transformed into a set of Boolean ranges like $[15^\circ, 20^\circ)$ or $[20^\circ, 25^\circ)$, allowing the machine to simply match the value.

Rule construction The mechanism for building a TM involves organizing Tsetlin Automata into teams, where each team is responsible for constructing a single conjunctive clause (AND-rule). These rules follow a logical pattern:

if condition then class.

For every potential literal in the input set (both original features and their negations), an assigned automaton determines whether to “Include” or “Exclude” that literal based on

its internal state [5]. By combining these decisions through the AND-operator, the team produces a logical rule. For the vehicle classification task described in [7], a resulting rule can be expressed as:

if *Four Wheels* **and** *Transports People* **then** *Car*.

Class prediction by voting The TM, as shown in Figure 2.1, feeds an input vector $X = (x_1, \dots, x_n) \in \{0, 1\}^n$ to several conjunctive clauses for evaluation. The number of clauses used is a user-defined parameter N . The clauses are assigned into two groups based on polarity: positive clauses (C_j^1 , $j \in \{1, \dots, N/2\}$) vote in favor of the target class ($y = 1$), negative clauses (C_j^0 , $j \in \{1, \dots, N/2\}$) vote against it ($y = 0$) [5].

While a standard clause learns to capture sub-patterns in X and outputs the conjunction of its literals, the TM uses a specific mechanism for empty clauses ($L_j = \emptyset$). These clauses without literals are not required for classification and can be pruned from the inference structure, but during learning, they remain active:

$$C_j(X) = \begin{cases} 1, & \text{if } L_j = \emptyset \text{ during learning,} \\ 0, & \text{if } L_j = \emptyset \text{ during classification,} \\ \bigwedge_{l_k \in L_j} l_k, & \text{if } L_j \neq \emptyset. \end{cases}$$

The final classification $\hat{y}$ is reached through a majority vote. The clause evaluations are combined into a final output through summation and unit step thresholding:

$$\hat{y} = u \left(\sum_{j=1}^{N/2} C_j^1(X) - \sum_{j=1}^{N/2} C_j^0(X) \right)$$

where $u(v) = 1$ **if** $v \geq 0$ **else** 0. The summation allows the model to weigh positive against negative evidence found in the input X . The example classifier that captures the XOR-relation can be represented as $\hat{y} = u(x_1\bar{x}_2 + \bar{x}_1x_2 - x_1x_2 - \bar{x}_1\bar{x}_2)$ [5].

Figure 2.3 illustrates the TM configuration featuring two clauses: one with positive polarity and one with negative polarity. These clauses are determined by Tsetlin Automata, which select the literals to include from the set $L = \{x_1, x_2, \bar{x}_1, \bar{x}_2\}$. In this example, the positive clause results in the form $x_1\bar{x}_2$, while the negative clause forms x_1x_2 .

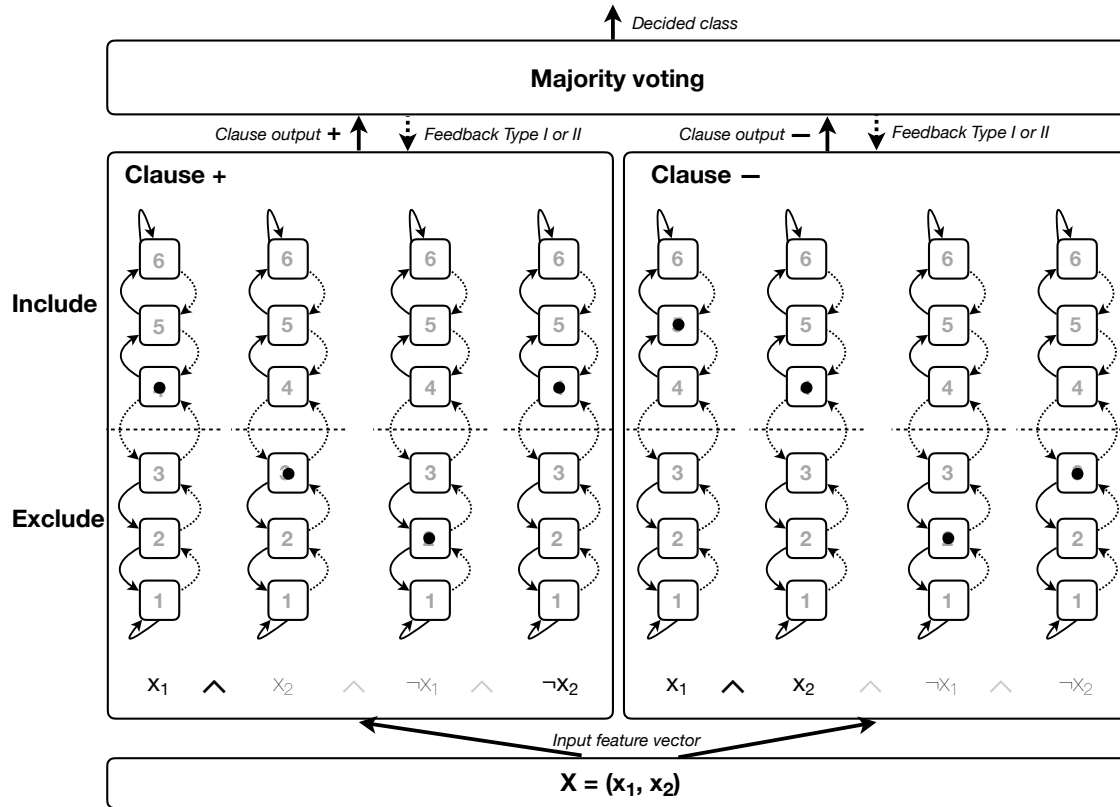


Figure 2.3.: Two Tsetlin Automata teams, each producing a conjunctive clause (source: [5])

2.4. Learning Algorithm

The learning mechanism of the TM functions through a game-theoretic approach, orchestrating Tsetlin Automata to maximize pattern recognition accuracy [5]. We follow the example found in [6] to demonstrate this process.

IMDb review sentiment example To demonstrate the learning process, we consider the task of classifying the sentiment of movie reviews from the IMDb dataset. This problem uses a Set of Words (SoW) representation, where every word in the vocabulary acts as an input feature. Consider the review “The movie was good, and I had popcorn.”. This text is converted into a binary input vector X . The inputs for words present in the review (e.g. “good” and “movie”) are set to True, while the inputs for any missing words are set to False.

Memory Consider the clause designed to detect positive sentiment targeting the pattern “good” AND “movie”. To manage this rule, the machine uses a team of Tsetlin Automata, assigning one automaton to track each potential literal. Figure 2.4 illustrates the memory states maintained by this team. Feedback from the learning process increases these states to remember the literal longer when a reinforcing truth value is observed, and without such observations, the values decrease, causing the clause to eventually forget the literal.

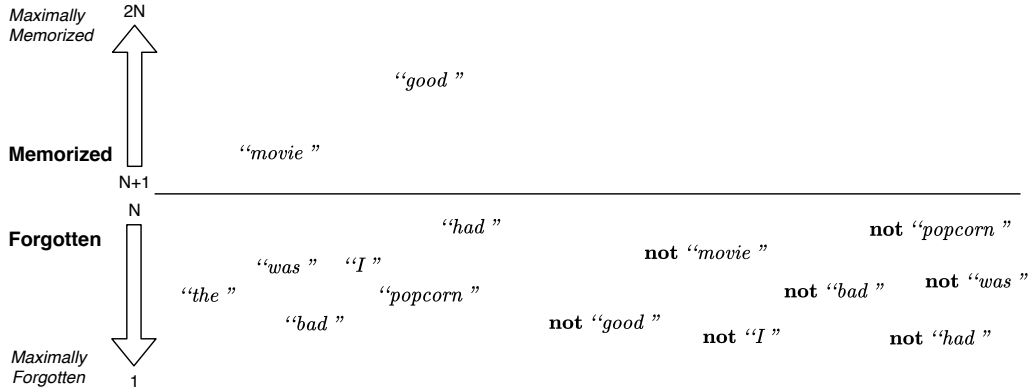


Figure 2.4.: Sticky memory for IMDb example with depth N for the clause: “good” AND “movie” (source: [6])

Memory updating Memory is updated on a specific input X to the TM. A user-defined parameter s controls how quickly truth values are forgotten. In result, increasing s makes the patterns finer (more specific), while decreasing s makes them coarser (more general). The process of updating the memory of each clause is handled by these three operators [6]:

- The **memorize**(X, s) operator handles **Type Ia feedback** (Figure 2.5). It strengthens the memory of an input X by increasing integer values for truth values present in X . Simultaneously, truth values that conflict with the input have their integers decreased randomly with probability $\frac{1}{s}$.
- The **forget**(s) operator handles **Type Ib feedback** (Figure 2.6). It performs forgetting by randomly decreasing the memory integer of all truth values with probability $\frac{1}{s}$. This gradually moves all literals towards the “maximally forgotten” state.
- The **invalidate**(X) operator handles **Type II feedback** (Figure 2.7). It adjusts the clause to reject the input X by increasing the memory integer of all False truth values. For instance, if the word “bad” appears in a negative review that was incorrectly classified, the system increases the integer for **not** “bad”.

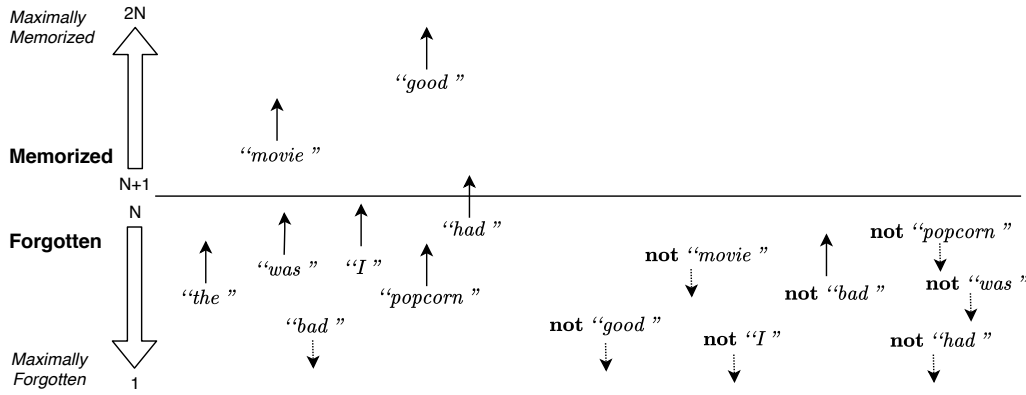


Figure 2.5.: Type Ia feedback triggered by input “The movie was good, and I had popcorn.” (source: [6])

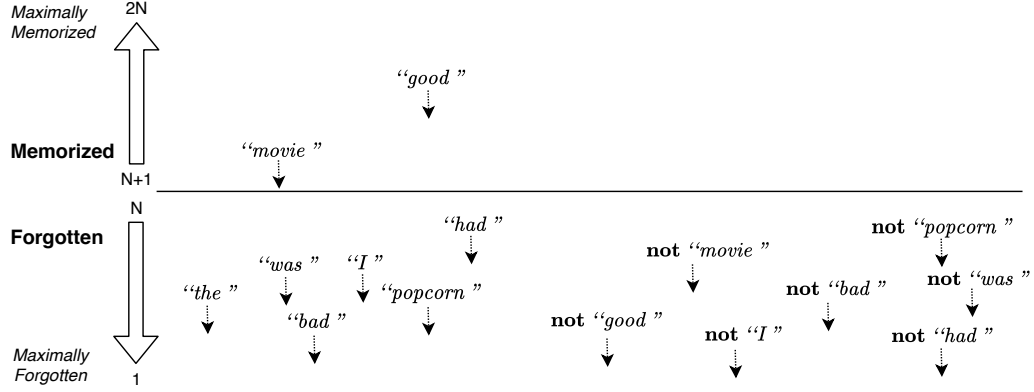


Figure 2.6.: Type Ib feedback triggered by input “Was good, and I had popcorn.” (source: [6])

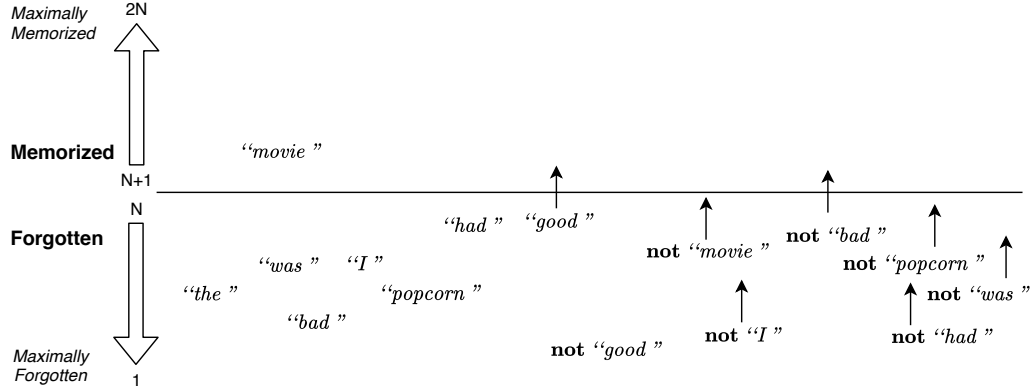


Figure 2.7.: Type II feedback applied to input “The movie was bad, and I had popcorn.” (source: [6])

Learning single clauses Prediction outcomes guide the learning of each TM clause [6]:

- **True Positive:** The clause correctly votes for its assigned output. In this case, the clause performs **memorize**(X, s). This feedback makes the clause remember and refine the pattern it recognizes in X .
- **False Negative:** The clause fails to vote for its assigned output. In this case, the clause performs **forget**(s). This reinforcement coarsens infrequent patterns, making them frequent.
- **False Positive:** The clause incorrectly votes for its assigned output. The clause then performs **invalidate**(X). This feedback makes the clause more discriminative.
- **True Negative:** The clause correctly refrains from voting for its assigned output. This

outcome does not trigger any memory updates.

Learning multiple clauses The TM achieves coordination by introducing another parameter, a voting margin t . This ensures that a sufficient number of clauses support each output without using more clauses than necessary. Learning multiple clauses works as follows [6]:

1. **Input:** Consider a training sample of truth values X and the target truth value y .
2. **Vote Aggregation:** Compute the net voting sum v by aggregating the output of all clauses that evaluate to *True* on input X . Votes in favor of the target class y are positive, while votes in favor of **not** y are negative. The final sum v is clipped to the range $[-t, t]$.
3. **Stochastic Update:** A feedback update for each clause is triggered with probability $P_{\text{update}} = \frac{t-v}{2t}$. If triggered, the clause is modified as follows:
 - a) **True Positives (Type Ia feedback):** Perform **memorize**(X, s) (the clause matches X and outputs y)
 - b) **False Negatives (Type Ib feedback):** Perform **forget**(s) (the clause does not match X and outputs y)
 - c) **False Positive (Type II feedback):** Perform **invalidate**(X) (the clause matches X and outputs **not** y)

XOR-gate example Figure 2.8 illustrates the learning process for an XOR-gate training sample with input $(x_1 = 0, x_2 = 1)$ and target output $y = 1$. Initially (Step 1), the positive clauses C_1 and C_3 do not match the input because they contain conflicting literals (e.g. $\overline{x_2}$ in C_3 conflicts with $x_2 = 1$). This results in a false negative prediction, which triggers Type Ib feedback. This feedback forces the clause to forget the conflicting literals. Once the clauses are generalized enough to match the input, Type Ia feedback reinforces inclusion of valid literals present in the input (e.g. x_2 in C_3). By Step t , these updates modified Clause C_1 to $\overline{x_1}x_2$ and C_3 to just x_2 allowing the machine to match the input and produce the correct prediction $\hat{y} = 1$.

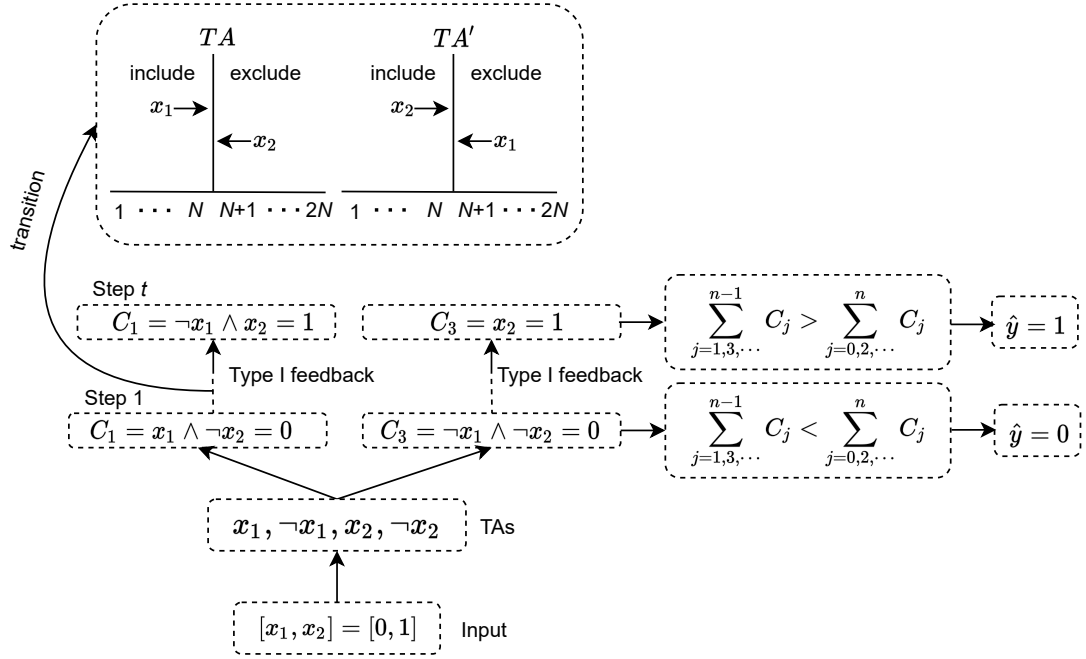


Figure 2.8.: TM learning dynamics for an XOR-gate training sample, with input $(x_1 = 0, x_2 = 1)$ and output target $y = 1$ (source: [11])

2.5. Tsetlin Machine Architecture Variants

Due to the modular design of the original TM, there is an ongoing effort to further develop this method. Researchers have implemented new iterations to address and mitigate limitations discovered in the original design. In this section, we briefly discuss three significant architectural variations and what they attempt to accomplish.

The Coalesced Tsetlin Machine The standard design of the TM supports prediction for only a single binary output. Multi-output problems require a workaround where a separate machine is created for every output. This approach in which each machine works independently limits the ability to share patterns that may be relevant to multiple classes. The Coalesced TM [6] introduces a shared pool of clauses. This is accomplished by a weight matrix that connects every clause to every possible output. Instead of a rule belonging to just one class, it is assigned an integer weight: a positive weight votes in favor of a class, while a negative weight votes against it. Empirical results on benchmark datasets show that this approach obtains significantly higher accuracy in 50 to 1000-clause configurations, indicating an ability to reuse clauses and stronger robustness against imbalanced training data [6]. Figure 2.9 shows an inference structure with three outputs, a newly introduced clause memory matrix and a clause weight matrix.

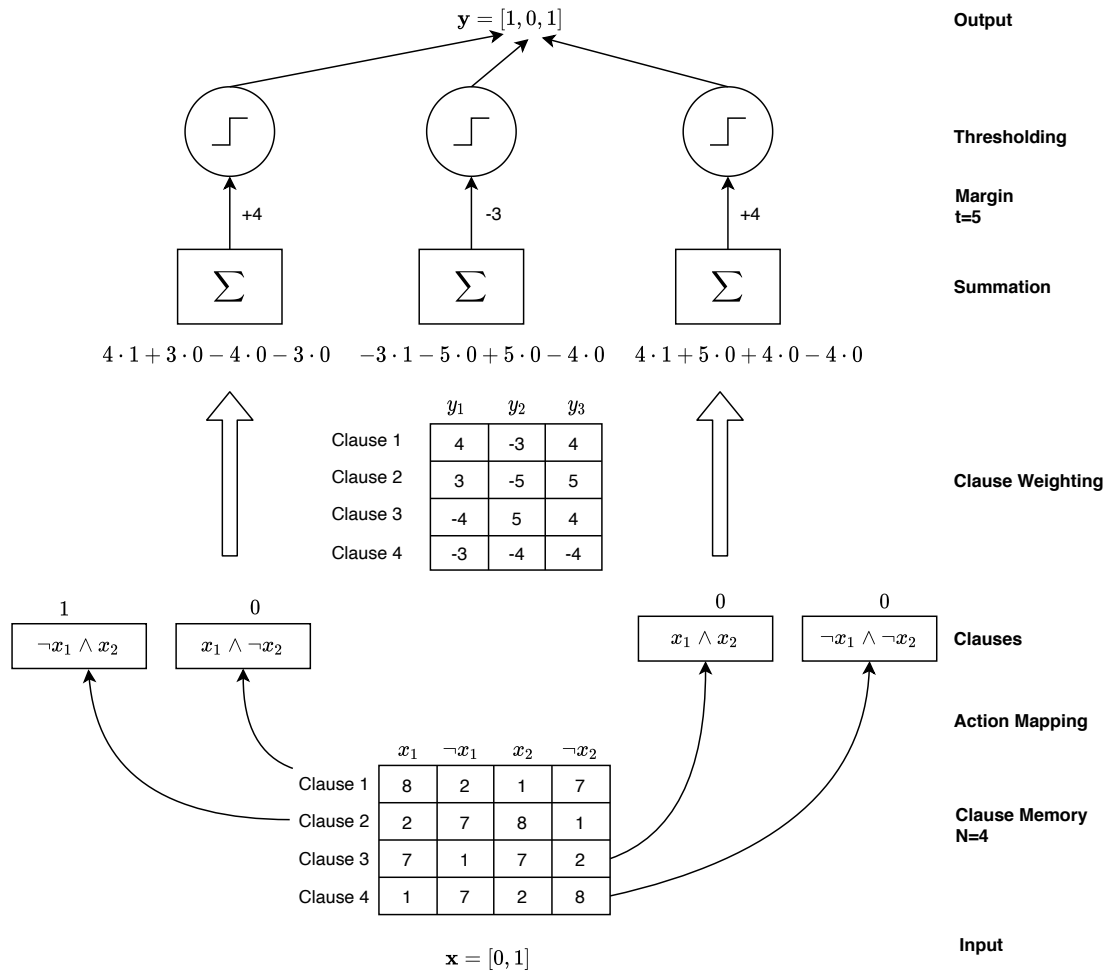


Figure 2.9.: The Coalesced TM structure with inference example (source: [6])

The Sparse Tsetlin Machine The standard TM maintains the entire state space, storing a significant number of unnecessary values that consume memory and computational time. The Sparse TM [12] addresses this by implementing the *Active Literals* mechanism, which focuses resources only on actively contributing literals. To achieve this, the architecture utilizes the Compressed Sparse Row (CSR) format and discards negated features, assuming that any missing feature is implicitly negated. A lower-bound threshold is added to ensure the compactness of the model, where any literal state reduced below the limit is removed from the clause. This significantly reduces memory footprint and computational time while maintaining competitive classification performance [12]. Figure 2.10 illustrates the feedback interaction between *Active Literals* and sparse clauses. During Type Ia feedback (reinforcing true positives), *Active Literals* identify and store meaningful literals, while Type II feedback (combating false positives) introduces these literals into the sparse clauses to invalidate incorrect matches.

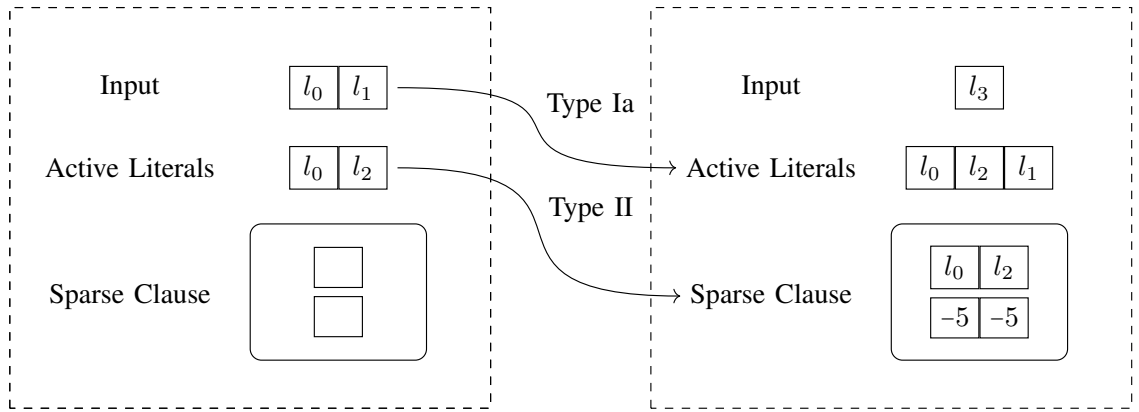


Figure 2.10.: The Sparse TM using Active Literals (source: [12])

The Stateless (Inference Only) Tsetlin Machine There is a major difference between how the TM learns and how it makes predictions. During the learning phase, the machine relies on a detailed memory state to track pattern confidence, storing a state integer (ranging from 1 to $2N$) for every potential literal. However, once training is completed, this detailed history is no longer needed to make predictions. For inference, the model only needs to know which specific literals to Include and which to Exclude, converting complex memory states into simple binary rules. By discarding learning memory and keeping only the final compiled rules, the model significantly reduces memory usage during deployment when on-device learning is not required. For example, in Figure 2.4, the inference model would discard the forgotten values and store only the final rule: “good” AND “movie”.

2.6. Comparison with Neural Networks

Neural Networks are currently the standard approach for AI applications, making them the common benchmark for new methods. For a direct theoretical comparison, the research conducted in [11] introduced the Weighted TM. This architecture assigns an integer weight to each clause based on its prediction accuracy, operating similarly to the Coalesced TM (Section 2.5) but without the clause sharing mechanism. With this modification, the TM behaves as a special case of a perceptron using logical clauses as binary inputs instead of raw features [11].

The difference in the decision boundaries on the Iris dataset is illustrated in Figure 2.11, where each model was trained for 50 epochs. The TM (Figure 2.11a) used 50 clauses and the Single Layer Neural Network (Figure 2.11c) utilized 50 neurons. The TM builds its decision boundaries from the binary output of individual clauses ($C_i = \{0,1\}$). It results in steps formed by filling contour lines rather than the smooth curves generated by neural networks. This difference changes how each model adapts to new data. In the TM, a clause (or a set of clauses) can simply learn the new pattern and create a step in the decision boundary to include the new data point in the correct region [11].

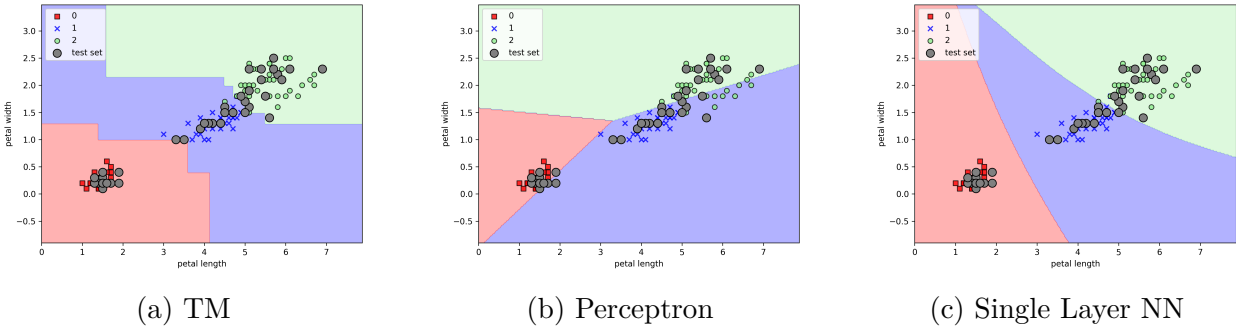


Figure 2.11.: Decision boundary visualization on the Iris dataset (source: [11])

Transparent logic Neural networks are often described as “black boxes” because their decision making logic is hidden and difficult for humans to interpret. This contrasts with the TM, where interpretability comes naturally from its logic-based approach. The produced rules are human-readable clauses (e.g. *if X and Y then Class A*), making it technically possible to even manually add or edit rules. This transparency allows users to examine the inner workings in applications where understanding the reasoning behind a prediction is mandatory.

Computational efficiency and optimization An important difference between the architectures lies in their underlying arithmetic. Neural networks typically rely on floating-point

matrix multiplications that require substantial computational resources. In contrast, the TM operates primarily using bitwise operators (AND, OR, NOT) and simple integer arithmetic. Theoretically, this offers efficiency advantages, enabling implementations on hardware lacking dedicated floating-point units. However, modern processors such as GPUs and TPUs are specifically engineered to accelerate matrix operations, since they are widespread in fields ranging from physics simulations to deep learning. As a result, the performance advantage of an integer-based approach is diminished on hardware that implements these optimizations.

Performance with limited data According to research presented in [5], TMs generalize better with smaller datasets compared to neural networks. In benchmarks such as the Noisy XOR problem, the TM maintained much higher accuracy on smaller datasets, whereas neural networks required larger datasets to converge. This makes the TM a suitable candidate for environments where data is limited or noisy.

2.7. Summary

In conclusion, the Tsetlin Machine is a logic-based alternative for pattern recognition that utilizes Boolean algebra. It is built upon Tsetlin Automata, which are finite-state machines that learn optimal actions through rewards and penalties. These automata are arranged into teams to construct “If-Then” rules known as conjunctive clauses, which identify sub-patterns in data. Then these clauses are for or against a target class to reach a final decision. The model is trained using a game-theoretic algorithm that employs different types of feedback to memorize, forget, or invalidate patterns. Compared to traditional “black box” neural networks, the Tsetlin Machine benefits from transparent logic and efficiency, both with computational resources and training data. Finally, recent architectural variants such as Coalesced, Sparse, and Stateless Tsetlin Machines address limitations of the original design.

3. Functional Scope

This chapter outlines the considerations and requirements for the library and the supporting tools. Describes the intended audience and details the key decisions made during development. These choices are further described in subsequent sections on Functional and Non-functional requirements.

3.1. Expected User Groups

Due to the experimental nature of this work, the key design goal was to make the library both easy to use and easy to understand. This is necessary as we anticipate that users may need to adapt the library for integration into their specific applications.

Embedded developers The library was designed with the limitations of embedded hardware in mind. Unlike many alternatives, it supports every step of the process on-device, while others implement predictions on-device only. Thanks to the designed model exchange format, users can choose to train models on faster hardware or use the *green_tsetlin* [1] library to later move them to the edge device. Additionally, the efficiency gains promised by TMs [5] are highly beneficial for these resource limited devices, where standard machine learning methods are often too expensive or complicated to run.

Researchers As TMs have only recently started to gain recognition, we expect researchers familiar with classic machine learning models to benefit from integrating them into their workflows. By offering a familiar interface, our library enables rapid prototyping. This distinguishes our solution from others that require deeper domain knowledge, making it easier to benchmark the models in various use cases.

3.2. Functional Requirements

On-device training and inference Our solution was designed to support both TM training and inference directly on target hardware. Unlike other embedded machine learning frameworks that perform only inference on edge devices, this library enables the model to learn

directly from newly gathered data. This is necessary for applications that adapt to local sensor data without a constant connection to the central server. It should however facilitate an inference only mode with a lighter model serialization format specifically for more resource constrained systems.

Interoperability and serialization The library must implement functionality for saving and loading model states. This requires a standardized serialization format to ensure compatibility with the *green_tsetlin* [1] library. This compatibility is essential for the users to leverage high-performance workstations to pre-train models, export them, and then deploy them directly to the embedded devices. Furthermore, models trained or tuned on the edge must also be exportable for backup or analysis on faster hardware.

Configurable hyperparameters Exposing TM model hyperparameters is necessary for fitting the model to its application with varied limitations dictated by hardware or the input data itself. Researchers and developers must be able to easily fine-tune these parameters when designing their solution for each specific problem. This allows optimization of performance, memory footprint, or accuracy without modifying library code.

Correctness with the theoretical work The implementation must behave as described in theoretical research [5]. It is required to accurately model the Tsetlin Automata logic and implement the specific learning rules (Type I and Type II feedback) defined in the original papers. This ensures that the software is a close representation of the method proposed in the theory and avoids unintended assumptions.

3.3. Nonfunctional Requirements

Portability The library must be designed for compatibility, avoiding dependencies on any specific hardware or operating system. The functionality of the external library should be defined during compilation if there is a risk that it may not work on the desired target. This strategy ensures that the code can be compiled with minimal, if any, modifications for a wide range of hardware architectures. The resulting portability, coupled with the adjustable model parameters, allows end users to create a solution that is both widely adoptable and future-proof.

Performance and memory usage Optimizations are significantly beneficial, as the library's performance impacts both the training step and, more importantly, the real-time predictions on target devices. Given that TMs heavily rely on binary logic and random number generators, optimizing these areas allows for better utilization of limited hardware resources.

This optimization extends to memory management, since the memory is needed to store the model on the device, as well as the memory used during runtime, must also be considered.

Simple API The Applications Programming Interface (API) must be intuitive and well-documented. Its design should follow standard interfaces developed by major machine learning libraries, allowing users unfamiliar with TM theory to quickly prototype concepts. This simple interface allows a developer to easily integrate the library into an existing project. Moreover, supporting quick prototyping in environments like Python Jupyter Notebooks alongside established models from libraries such as Scikit-learn speeds up development before deployment to embedded devices.

Testing and examples As with any software project, testing enhances the development process and maintenance. Code examples provide a starting point for users unfamiliar with the project. Furthermore, simple installation and easy to run examples encourage potential users to quickly try the library. Finally, these tests help verify that the library runs correctly on new devices with different hardware limitations.

4. Selected Realization Aspects

This section describes the design choices of *tsetlin_machine_c* as well as the rationale behind them, and the overarching architecture of the solution. The actual TM implementation is described in detail to make it intuitive. It describes used tools, solutions and algorithms. Additionally, it discusses several interesting problems that were encountered.

4.1. System Architecture and Data Workflow

The project's architecture and data flow are shown in Figure 4.1. The design is split between a **host (PC)** and an **embedded device**, allowing models to be developed on one platform and transferred to different hardware for deployment.

Training and exporting The process starts by defining and training TM models using an external library (**C**). In this project, we use the *green_tsetlin* [1] library. Once training is complete, our Python Interface (**G**) handles exporting the model. We support two different interchange formats:

- FlatBuffers [13] (**D**) - An efficient, cross-platform serialization format utilizing a third-party library.
- Custom Binary (**E**) - A dependency-free format that may require adjustments based on the target hardware.

On-device execution After the serialized model state (**I**) is saved, it can be moved to the embedded device. On the device, we use our core library (**O**), which supports every step of the machine learning workflow:

- It can load a pretrained model (**K**) to start making predictions (**N**) immediately.
- It can be trained further on the device (**L**) using data gathered during operation (**P**).
- It can save and export the updated model state (**M**) to back up or inspect what the device has learned.
- It also supports creating new models (**J**) directly on the device if needed.

Python integration Since the core library is self-sufficient, it can also be compiled for the host architecture. This allows us to provide a Python package with bindings to the same C code, so that it can run on a standard computer. This package includes a *Scikit-learn*-like

interface (F) and is easy to install, similar to other common machine learning tools. This enables rapid testing of ideas in *Jupyter Notebooks* (H) and makes it possible to benchmark our implementation against standard models provided by Python libraries.

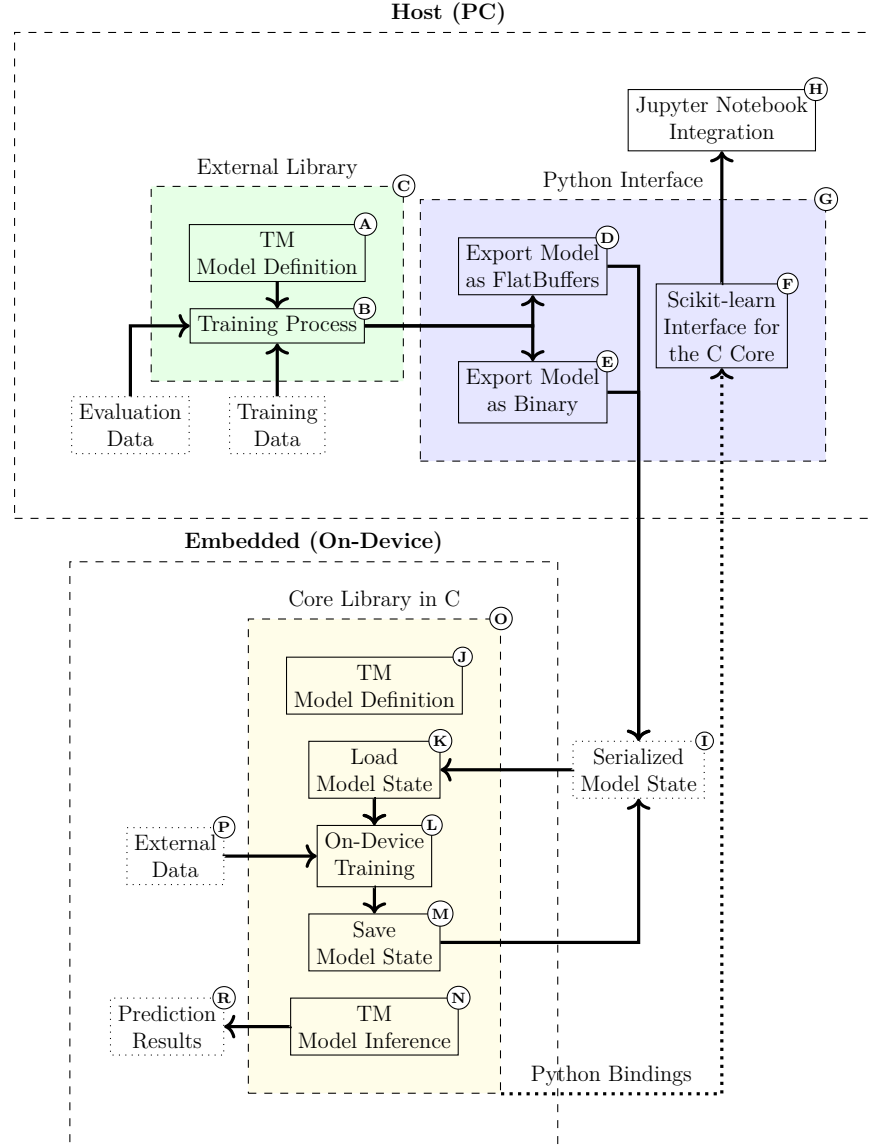


Figure 4.1.: System architecture and data workflow. Models can be pretrained using *green_tsetlin* [1] library (C). We provide a Python Interface (G) that handles model serialization and includes a Scikit-learn compatible wrapper. Our Core TM Library (O), implemented in the C programming language, executes the model on the target device, supporting both inference and on-device training.

4.2. Design

Even though optimization was the primary objective, not every method could be utilized due to the target environments having very limited resources and hardware limitations. In particular, vectorization was completely out of the picture, even though *green_tsetlin* [1] has it enabled by default. GPU programming was obviously dismissed for the same reason – although notably, such an implementation is possible [4].

4.2.1. Programming Language

Since the thesis topic requires *tsetlin_machine_c* to be designed for embedded devices, there were only a few suitable options like C/C++, Rust or MicroPython. C was chosen as the most widely used language for embedded systems – it is efficient, lightweight and portable. Its popularity also means it has extensive third party library support. The authors' previous experience working with C/C++ also influenced this decision. Exporting models from *green_tsetlin* [1] is done in Python, as it is a Python library. *tsetlin_machine_c* also has Python bindings so that it can be used conveniently from Python.

4.2.2. Execution Engine

Even though there exists a number of existing libraries implementing the Tsetlin Machines, *tsetlin_machine_c* was essentially created from the ground up without taking much inspiration or hints from them. It was required to be written primarily with optimization in mind so that it works as fast as possible while maintaining small size and portability on top of utilizing little power and memory.

It was also decided *tsetlin_machine_c* should support multiple different TM variants so their efficiency could be tested – Coalesced, Sparse and Stateless.

4.2.3. Model Serialization Library

Several libraries were compiled and compared as possible options to implement the model serialization format. Only C libraries were taken into consideration. See Table 4.1.

Table 4.1.: Comparison of selected model serialization libraries

Format name	Comment	Schema complexity	Efficiency on edge device	Library size on edge device
Protocol Buffers	Simple and small model representation	Low	Fast but needs object parsing	Too big
ONNX	A standardized way to store models and operation graphs	High	Low - needs graph and tensor parsing	Much too big
FlatBuffers	“Protocol buffers lite”	Low	Very fast, zero-copy	Acceptable

4.3. TM implementation

This section describes the implementation of the different TM types. Each type is broken down into its main components, and provides technical intuition about their operation.

4.3.1. Tsetlin Automaton

A Tsetlin Automaton is the smallest part of a TM, and it is what clauses consist of. Implementation-wise it is represented by a singular, 8-bit integer, meaning its values can range from -127 to 127 by default. The range can be made smaller by setting model parameters max_state and min_state . Further on in this thesis, the term that a Tsetlin Automaton is active, means its internal integer value is greater or equal than halfway between max_state and min_state ; $value \geq \frac{max_state + min_state}{2}$. One input bit on position i has two corresponding Tsetlin Automata at positions in clause $2 \times i$ and $2 \times i + 1$. The first one – positive – decides whether the input bit – x_i – should be included in the clause, and the other – negative – whether its negation – $\neg x_i$ – should be included.

4.3.2. Coalesced TM

Coalesced TM is the variant most closely resembling the original TM out of those implemented by *tsetlin_machine_c*. The implementation closely follows the paper [6].

Training

The training loop consists of three distinct steps repeated for each datapoint, further explained below.

1. Calculate clause output.
2. Sum votes.
3. Calculate and apply feedback.

Calculate Clause Output The current states of Tsetlin Automata in each clause are used to determine whether the clause activates – 1, or not – 0. This output is temporarily stored in an internal array. For this step specifically, a clause is active if either of two conditions is met:

- **The clause is empty** – no Tsetlin Automaton is active.
- For the training input datapoint, all bits corresponding to active positive Tsetlin Automata of this clause are set and all bits corresponding to active negative Tsetlin Automata of this clause are unset.

Sum Votes This step uses the information calculated in the previous point whether clauses are active or not. Active clauses vote for all possible classes with trained weights. Specifically, clause i votes for class j with trained weight $w_{i,j}$. $w_{i,j}$ can be positive or negative. As all active clauses vote, each class accumulates their votes. The sum of votes is then clipped to parameter *threshold*. It serves the same purpose as gradient clipping in other ML architectures, mainly for numerical stability.

Calculate and Apply Feedback As the sum of votes indicates whether a class should be chosen or not, that prediction can be compared to the actual datapoint label. Based on this information, a suitable feedback to be applied to Tsetlin Automata. For each datapoint two classes are specified for feedback:

- Positive – one of the classes, or the one class, specified by label.
- Negative – one of the classes not specified by label.

Both have a chance to trigger feedback to all clauses. For the positive class, the chance is inversely proportional to the clipped sum of votes it received. That means the model applies feedback more often the *less* sure it is the class is correct. Although unintuitive at first, this practice is used to avoid overfitting. For the negative class, the chance is proportional to the clipped sum of votes it received. This means the model applies feedback more often the more sure it is the class is incorrect.

What is unique to *tsetlin_machine_c*, it allows both predictions and output to be customized to be any type of data. A custom feedback function can be set for the model, to override how positive and negative classes are chosen. The default uses normal classification, where the output and prediction is an integer identifying a class. Another predefined function

allows multi-class classification, where output and prediction is an 8-bit integer array where the length is the number of classes, where value 1 signifies a class with a given identifier being chosen, value 0 – not chosen. Also a custom comparison function can be set for the model, to override when the output and prediction data classes should be treated as equal. By default a simple memory comparison is used, which handles the vast majority of uses – even when fully customized.

Feedback When a chosen class triggers feedback to all clauses, for each clause separately one of three feedback types may be applied. Tsetlin Automata changes specified in **Action** are not always changed – they have a random chance to do so based on the parameter s – sensitivity – further explained in **Sensitivity** for each feedback type.

- **Type Ia**
 - **Condition:** The clause is active and voted correctly for the chosen class (with the correct weight sign).
 - **Action:** Reinforce Tsetlin Automata making up the clause, reinforce weight for the chosen class.
 - **Sensitivity:** The chance for change is either $\frac{1}{s}$ for true positive and true negative Tsetlin Automata, and otherwise its inversion $-\frac{s-1}{s}$. Model parameter *boost_true-positive* makes it so that true positive Tsetlin Automata are always changed.
 - **Intuition:** The clause should continue to vote for the same class.
- **Type Ib**
 - **Condition:** The clause is inactive and (would have) voted correctly for the chosen class.
 - **Action:** Lower the confidence of all the Tsetlin Automata making up the clause, positive and negative.
 - **Sensitivity:** The chance for change is always $\frac{1}{s}$.
 - **Intuition:** The clause should “find something else to do”, countering force.
- **Type II**
 - **Condition:** The clause is active and voted incorrectly for the chosen class.
 - **Action:** Raise, towards activation, inactive Tsetlin Automata that could deactivate the clause for this datapoint, punish the weight of the chosen class towards zero.
 - **Sensitivity:** Change always happens.
 - **Intuition:** Try two things at once: exclude the clause and fix the weight (flip the sign). Although these goals are opposing, just one of them needs to come true, and the hope is one of them should do so before the other.
- Nothing happens if the clause is inactive and would have voted incorrectly.

Inference

Inference on a datapoint looks very similar to training, with few key differences.

1. Calculate clause output
2. Sum votes
3. Apply output activation

Calculate Clause Output Unlike in training, an empty clause – with no active Tsetlin Automata – is inactive. Otherwise it does the same thing – calculates which clauses are active with a given input.

Sum Votes Exactly the same as training. Sums up class votes from active clauses.

Apply Output Activation Applies a function to determine the output from the summed up class votes. By default, it simply returns an integer – an identifier of a class with the most votes.

Again, this can be fully customized. A custom output activation function can be set for the model, to override what the model outputs based on the summed up weights.

Another predefined function allows multi-class classification, where the output is an 8-bit integer array where the length is the number of classes, where value 1 signifies a class with a given identifier being chosen, value 0 - not chosen.

Data Layout

A clause is represented as a simple array of Tsetlin Automata states. For the sake of efficiency they are ordered by the literals they correspond to, in pairs of positive and negative Tsetlin Automata. This simplified representation is visualized in Figure 4.2.

The i -th array field represents the state of Tsetlin Automaton corresponding to literal with index $\lfloor \frac{i}{2} \rfloor$ – positive if i is even, negative if odd.

Index:	0	1	2	3	...	$2 \times \text{num_literals} - 2$	$2 \times \text{num_literals} - 1$
Corresponding literal:	x_0	$\neg x_0$	x_1	$\neg x_1$	...	$x_{\text{num_literals}-1}$	$\neg x_{\text{num_literals}-1}$
TA state:	13	-43	-48	-12	...	17	-5

Figure 4.2.: Clause data layout of Coalesced TM

Model Serialization Format

The model state is represented by the FlatBuffers schema in Listing 4.1. It defines the model parameters, clause weights, and the internal states of the Tsetlin Automata.

Listing 4.1.: Coalesced TM FlatBuffers serialization format

```
1  table Parameters {
2    threshold: uint; // tsetlin automaton threshold
3    n_literals: uint; // total number of literals
4    n_clauses: uint; // total number of clauses
5    n_classes: uint; // number of output classes
6    max_state: byte; // maximum state for automaton
7    min_state: byte; // minimum state for automaton
8    boost_tp: ubyte; // boost true positive feedback
9    learn_s: float; // learning sensitivity (s)
10 }
11 table ClauseWeightsTensor {
12   weights: [short]; // clause weights
13   shape: [uint]; // dimensions e.g. [n_clauses, n_classes]
14 }
15 table AutomatonStatesTensor {
16   states: [byte]; // automaton states
17   shape: [uint]; // dimensions e.g. [n_clauses, n_literals, 2]
18 }
19 table Model {
20   params: Parameters (required); // model parameters
21   automaton_states: AutomatonStatesTensor (required); // automaton states
22   clause_weights: ClauseWeightsTensor (required); // clause weights
23   literal_names: [string]; // names of literals
24 }
25 root_type Model;
```

4.3.3. Sparse TM

A Sparse TM [12] aims to efficiently process sparse data. It uses the same core principles and mechanics as the Coalesced TM, its main difference being the underlying data layout. It also introduces *Active Literals* – it functions as a list of literals considered to be added to sparse state. Notably, both sparse state and *Active Literals* need to have a limited size

or the model is slower and larger than Coalesced TM, and learns poorly – something not mentioned in the paper [12].

Data Layout

Unlike the Coalesced TM, the Sparse TM does not store all Tsetlin Automata. Simply put, it only keeps the Tsetlin Automata, which state is above a certain threshold. By doing so, in principle it does not need to waste space and computation power for Tsetlin Automata it deems unlikely to become active for a given clause. The simplified structure is shown in figure 4.3

Since not all Tsetlin Automata are present in a clause a linked list is used as the underlying data structure. Identifying indices cannot be inferred from the data structure and need to be stored in nodes.

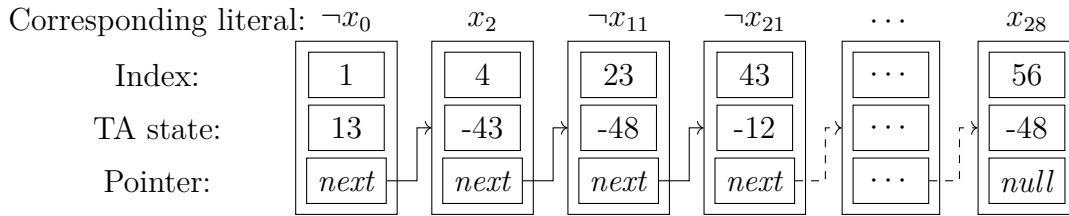


Figure 4.3.: Clause data layout of Sparse TM

Training

Aside from obvious changes from Coalesced TM needed to accommodate the different data structure, the feedback mechanism has some modifications. Namely, a new Tsetlin Automaton can be introduced to a clause only in type Ia of feedback, only when the corresponding literal currently belongs to Active Literals of a class which triggered this feedback. A literal can be added to *Active Literals* in feedback type II. *Active Literals* are not being explicitly removed, but the oldest is replaced when adding a new one.

4.3.4. Stateless TM

A Stateless TM is a slimmed down version of Sparse TM that does not allow any model training, at the same time further reducing model size to the smallest and fastest possible for inference. It should only be created from a trained model of a different TM variant.

Data Layout

Stateless TM, as the name suggests, does not store Tsetlin Automaton states at all. Here a degenerated version of Tsetlin Automaton is active if it is present at all. The simplified structure is shown in figure 4.4

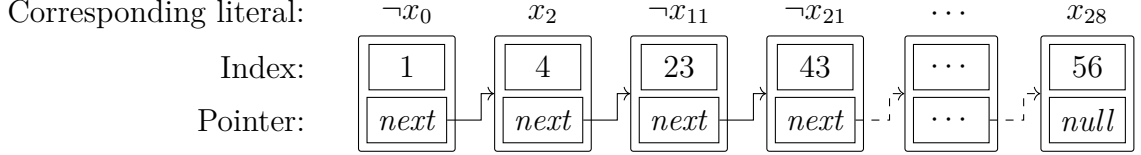


Figure 4.4.: Clause data layout of Sparse TM (Stateless)

4.4. Optimizations

Seeing as speed, size, portability as well as limiting the usage of power and memory were all key requirements, optimizations were the main focus of this project. In this section a few of the more interesting examples are listed.

4.4.1. Pseudo Random Number Generation

Analyzing the efficiency of a development version with valgrind and callgrind, it was obvious that most training time was spent on calls to `rand()` - pseudo random number generation. While it is not costly to call once, the cost quickly accumulates over millions of calls.

Analysis

`rand()` is costly because it aims for a uniform spread, as evenly as possible. `tsetlin_machine_c` intentionally adopts a design that relaxes this uniformity, accepting minor statistical deviations that average out over millions of calls – in order to prioritize execution speed and reduced power consumption. The pseudo random number generation for this purpose does not need to be cryptographically secure either – `rand()` is not anyway. A quick look at `green_tsetlin` [1] revealed that it suffered from the same problem. Its solution was to use a `wyhash64` algorithm. Using just an 64-bit integer addition, two 128-bit integer multiplications and few bitwise operations as well as a double multiplication, it is overall a much cheaper solution. Listing 4.2 shows the `green_tsetlin` [1] implementation of `wyhash64`.

Listing 4.2.: *green_tsetlin* [1] implementation of wyhash64

```
1 double next_u()
2 {
3     wyhash64_x += 0x60bee2bee120fc15;
4     __uint128_t tmp;
5     tmp = (__uint128_t) wyhash64_x * 0xa3b195354a39b70d;
6     uint64_t m1 = (tmp >> 64) ^ tmp;
7     tmp = (__uint128_t)m1 * 0x1b03738712fad5c9;
8     uint64_t m2 = ((tmp >> 64) ^ tmp);
9
10    double r = 0x1.0p-64 * m2;
11    return r;
12 }
```

Solution

Wyhash64 represents a significantly lower-cost solution compared to the alternative; however, its computational and energy costs remain non-negligible. 128-bit integer multiplications and double multiplication cost many precious CPU cycles which means time and power. Listing 4.3 shows the *tsetlin_machine_c* implementation of `xorshift32`. All that is really needed are a few bit shifts and XOR operations. It uses so few CPU cycles it is basically free. It might also be the case one of the authors likes low-level optimization an unhealthy amount.

Listing 4.3.: *tsetlin_machine_c* implementation of xorshift32

```
1  uint32_t prng_next_uint32(struct FastPRNG* prng) {
2      uint32_t x = prng->state;
3      x ^= x << 13;
4      x ^= x >> 17;
5      x ^= x << 5;
6      prng->state = x;
7      return x;
8  }
9
10 float prng_next_float(struct FastPRNG* prng) {
11     uint32_t x = prng->state;
12     x ^= x << 13;
13     x ^= x >> 17;
14     x ^= x << 5;
15     prng->state = x;
16     static union {
17         uint32_t u32;
18         float f;
19     } caster;
20     /* Reinterpret x as float: keep random mantissa, set exponent to 127
21        (1.0f), then subtract 1.0f to obtain value in [0,1).
22     */
23     caster.u32 = 0x3F800000 | x >> 9;
24
25     return caster.f - 1.0f;
26 }
```

4.5. Summary

In conclusion, *tsetlin_machine_c* is a C library designed for resource-limited systems, using FlatBuffers format for model serialization. It can be used conveniently from Python on the host device, both inference and training is available from C on the edge device, and models can be transferred between them – as shown in Section 4.1. This chapter furthermore touches on the actual, low-level implementation of TM types. Notable optimizations are also explained here.

5. Work Organization

This project combines elements of both research and development. Aside from implementing a functional TM execution engine, other goals were to test its effectiveness in different conditions and develop a lightweight model serialization format. It was also essential to choose the right tools for the task.

Since the topic of this thesis is not very well known, before starting development it was necessary to gain a deep understanding of the TM architecture. The learning materials as well as documentation for existing solutions were found to be lacking, inspiring *tsetlin_machine_c* to provide better documentation for its users, as well as in-code comments for anyone seeking deeper understanding.

The project followed an iterative methodology, cycling through stages of requirements definition, analysis, implementation, and testing. This approach was well-suited to the small team size and the research-oriented nature of the work, which made it difficult to estimate timelines or guarantee the feasibility of specific features from the beginning.

5.1. Team Members and Their Roles

The two team members worked on parts of the project based on their previous experience, specializations, and interests. The other person then reviewed it to the best of their ability and knowledge.

Igor Sikora

- C implementation of the TM execution engine
- research into possible optimizations and their implementation
- correctness tests
- code and user documentation

Robert Mesek

- Python bindings of *tsetlin_machine_c*
- integration with *green_tsetlin* [1]
- benchmarks, efficiency tests
- Jupyter Notebook examples
- build system with CMake

- integration with FlatBuffers for model serialization and deserialization

Naturally, many aspects of the project had to be realized together, such as solving architectural issues and making sure the TM execution engine implementation is correct with the theoretical work [5].

5.2. Used Tools

Git with GitHub Git is the most popular version control system, in this case used together with GitHub as a hosting service. This setup made it much easier to keep track of changes, and made it possible to work simultaneously on multiple issues on separate branches. Using GitHub also helped with transparency, since every update was visible to the whole team, also making reviews straightforward. The source code for this project is available in the public repository [14].

Discord Discord served as the main platform for day-to-day communication. Different channels could be used to separate topics, which helped keep discussions on-track and organized instead of everything being mixed in one place. Text chat was the most common, but voice calls were also used when something needed to be explained in more detail, or simply during working sessions. Since Discord is free and everyone was already familiar with it, there were no technical hurdles in getting started, and the communication stayed active throughout the project.

Email and MS Teams Communication with the project supervisor relied on email and MS Teams. Email was mainly for more formal messages such as documenting milestones. MS Teams was used for scheduled meetings and sharing files that needed feedback. It was helpful to separate internal group communication from supervisor communication, since it kept the workflow cleaner and avoided unnecessary message clutter.

Overleaf Overleaf was the tool used for writing and editing the L^AT_EX report. Using a shared online editor made collaboration much easier, making work in the same document possible without worrying about file versions. As it could be shared directly with the supervisor, no one had to wait for access to the latest version of the documents.

6. Technical Documentation

The *tsetlin_machine_c* library is a low-level C implementation of the TM. It is optimized for speed and size, making it suitable for environments with limited resources. It comes with a model serialization format designed to minimize model size and a way to convert existing *green_tsetlin* [1] models to this format. It is also possible to use *tsetlin_machine_c* conveniently from Python with the same syntax as many popular Machine Learning libraries thanks to its python bindings.

6.1. Case Study

For a quick start with *tsetlin_machine_c*, a usage example is provided – where initial training is conducted in Python, then model is transferred over to edge device where it can be used for inference and incremental training.

Listing 6.1 shows data preparation steps.

Listing 6.1.: Example Python data preparation steps

```
1 import numpy as np
2 import pandas as pd
3 from sklearn.model_selection import train_test_split
4 from sklearn.datasets import load_digits
5 from sklearn.preprocessing import OneHotEncoder
6 from sklearn.metrics import accuracy_score
7 from tsetlin_machine_py.c_tsetlin_clf import CTsetlinClassifier
8
9 # Generate the dataset with specified parameters
10 digits = load_digits()
11 X_data, y_data = digits.data, digits.target
12
13 # Convert the dataset to binary classification
14 X_data = OneHotEncoder(sparse_output=False).fit_transform(X_data)
15
16 # Split the dataset into training and testing sets
17 X_train_np, X_test_np, y_train, y_test = train_test_split(
18     X_data, y_data, test_size=0.2, stratify=y_data, random_state=42
19 )
20
21 # Convert NumPy arrays to Pandas DataFrames
22 feature_names = [f"feature_{i}" for i in range(X_data.shape[1])]
23 X_train = pd.DataFrame(X_train_np, columns=feature_names)
24 X_test = pd.DataFrame(X_test_np, columns=feature_names)
```

Listing 6.2 shows the fitting of the model. The model is initialized with default parameters.

Listing 6.2.: Example Python model fitting

```
1  # Fit the model
2  model = CTsetlinClassifier(random_state=42)
3  model.fit(X_train, y_train)
```

Listing 6.3 shows the evaluation of the model, as an optional step.

Listing 6.3.: Example Python model evaluation

```
1  # Predict on the test set
2  y_test_pred = model.predict(X_test)
3
4  # Calculate Accuracy
5  test_accuracy = accuracy_score(y_test, y_test_pred)
6  print(test_accuracy)
```

Listing 6.4 shows the export of the model to FlatBuffers format.

Listing 6.4.: Example Python model export to FlatBuffers format

```
1  # Save the model to FlatBuffers format
2  model.save_model("path/to/file")
```

Listing 6.5 shows the import of the model in C.

Listing 6.5.: Example C model import

```
1  #include <stdint.h>
2  #include <stdlib.h>
3  #include "tsetlin_machine.h"
4
5  uint32_t y_size = 1;
6  uint32_t y_element_size = sizeof(int32_t);
7  // Load in model
8  struct TsetlinMachine *tm = tm_load_fbs("path/to/file", y_size, y_element_size);
```

Listing 6.6 outlines the general steps to load data in C.

Listing 6.6.: Example C data loading

```
1  // Load in data
2  uint32_t rows = 70000;
3  uint32_t cols = 784;
4  uint8_t *x_data = malloc(rows * cols * sizeof(uint8_t));
5  int32_t *y_data = malloc(rows * sizeof(int32_t));
6  if (x_data == NULL || y_data == NULL) {
7      fprintf(stderr, "Failed to allocate memory for x_data or y_data\n");
8      return 1;
9  }
10 printf("Loading MNIST data\n");
11 load_mnist_data(x_data, y_data); // helper function to load data
12
13 uint8_t *x_train = x_data;
14 int32_t *y_train = y_data;
15 uint8_t *x_test = x_data + 60000 * cols;
16 int32_t *y_test = y_data + 60000;
```

Listing 6.7 shows the fitting of the model in C. The same method can be used to incrementally train the model, for example on each new datapoint.

Listing 6.7.: Example C model fitting

```
1 // Train model on 1000 train rows for 1 epoch
2 tm_train(tm, x_train, y_train, 1000, 1);
```

Listing [6.8](#) shows the evaluation of the model in C, as an optional step.

Listing 6.8.: Example C model evaluation

```
1 // Evaluate model on 1000 test rows
2 tm_evaluate(tm, x_test, y_test, 1000);
```

In C used structures have to be manually deallocated. Listing [6.9](#) shows this step.

Listing 6.9.: Example C deallocation

```
1 // Clean up
2 tm_free(tm);
3 free(x_data);
4 free(y_data);
```

Alternatively, a model instance can be created directly from C, instead of loading an existing model. Listing [6.10](#) shows such an example.

Listing 6.10.: Example C model initialization

```
1  srand(42);
2  // Set TsetlinMachine parameters
3  uint32_t num_classes = 10;
4  uint32_t threshold = 1200;
5  uint32_t num_literals = 784;
6  uint32_t num_clauses = 1000;
7  int8_t max_state = 127;
8  int8_t min_state = -127;
9  uint8_t boost_true_positive_feedback = 1;
10 uint32_t y_size = 1;
11 uint32_t y_element_size = sizeof(int32_t);
12 float s = 20.0f;
13 uint32_t seed = 42;
14 struct TsetlinMachine *tm = tm_create(num_classes, threshold, num_literals,
    ↪ num_clauses,
15     max_state, min_state, boost_true_positive_feedback, y_size, y_element_size,
    ↪ s, seed);
16 if (tm == NULL) {
17     perror("tm_load failed");
18     return 1;
19 }
```

6.2. API

The three main components of the *tsetlin_machine_c* library are the *TsetlinMachine*, *SparseTsetlinMachine*, and *StatelessTsetlinMachine* classes.

Data layout conventions used across the API:

- **X**: Flat array of shape (rows, num_literals) of uint8_t (each value 0 or 1).
- **y**: Flat array of shape (rows, y_size) with element size y_element_size.
- **y_pred**: Flat array of shape (rows, num_classes) with element size y_element_size.

6.2.1. TsetlinMachine

The basic TM variant.

Parameters:

- **num_classes**: Number of classes to predict.
- **threshold**: Threshold for clause votes.
- **num_literals**: Number of literals (features) in the input data.
- **num_clauses**: Number of clauses in the TM.
- **max_state**: Maximum automaton state value.
- **min_state**: Minimum automaton state value.
- **boost_true_positive_feedback**: Whether to boost true positive feedback (0/1).
- **y_size**: Size of the output vector (e.g., 1 for single-label classification).
- **y_element_size**: Size of each output element (in bytes).
- **s**: Sensitivity / learning parameter ($s > 1.0$).
- **seed**: Random seed for reproducibility.

Advanced Usage

This implementation is highly versatile; both input and output may come in any shape. The only limiting factors are that the input is binary and it is a classification architecture (as opposed to regression). There are two presets available:

1. **class_idx**: The output is a 32-bit unsigned integer signifying the index of chosen class.
2. **bin_vector**: The output is an array of 8-bit binary values, where 1 signifies a chosen class.

Custom formats can also be freely implemented. All that is required is to write the implementation based on the functions below and then set them as callbacks with the corresponding `tm_set_...`

6.2.2. SparseTsetlinMachine

A TM variant utilizing sparse memory layout. Its usage is very much symmetrical to the vanilla `TsetlinMachine`.

Advanced Usage

Similarly, custom formats can also be freely implemented. All that is required is to write the implementation based on the functions below and then set them as callbacks with the corresponding `stm_set_...`

6.2.3. StatelessTsetlinMachine

An inference-only TM variant utilizing sparse memory layout optimized for small size and inference speed.

Advanced Usage

Similarly, custom formats can also be freely implemented. All that is required is to write the implementation based on the functions below and then set them as callbacks with the corresponding `sltm_set_...`

6.3. Build

The build is automated with CMake.

Listing 6.11.: Project build commands

```
1 cmake -B build
2 cmake --build build
```

6.3.1. Build Requirements

- CMake 3.23+
- GCC-13 Compiler recommended
- uv Python manager (optional, for `green_tsetlin` integration and Jupyter Notebooks examples)

6.3.2. Build Options

- `-DBUILD_FLATCC=ON/OFF`: Build FlatCC from third_party
- `-DBUILD_PYTHON=ON/OFF`: Enable building Python bindings
- `-DBUILD_EXAMPLES=ON/OFF`: Enable building examples
- `-DBUILD_TESTING=ON/OFF`: Enable building third_party tests

6.4. Installation

To build the library into a shared object, the following commands can be run.

Listing 6.12.: Library installation commands

```
1 cmake -B build -DCMAKE_INSTALL_PREFIX=install -DBUILD_SHARED_LIBS=ON
2 cmake --build build
3 cmake --install build --component tsetlin_machine_c
```

6.4.1. Install with Python

To install the Python package and its dependencies, the following command can be run.

Listing 6.13.: Python environment synchronization

```
1 uv sync --extra green-tsetlin
```

6.5. Testing and Examples

To run accuracy tests, the following commands can be run.

Listing 6.14.: Running accuracy tests

```
1 cmake -B build
2 cmake --build build
3 ./build/tests/test_runner
```

Examples can be run as so:

Listing 6.15.: Running MNIST examples

```
1 uv sync
2 uv run python examples/mnist/python/get_data.py
3 ./build/examples/mnist/mnist_training
4 uv sync --extra green-tsetlin
5 uv run python examples/mnist/python/get_pretrained.py
6 ./build/examples/mnist/mnist_pretrained_inference
7 ./build/examples/mnist/mnist_model_size
```

7. Evaluation

This chapter evaluates the performance of our Tsetlin Machine implementation. To ensure comparability with existing research, we focus on standard datasets used in the original studies [5] (e.g. the Noisy-XOR problem). In addition, we provide a baseline comparison with established machine learning models from the Scikit-learn [15] library.

Environment The evaluation was conducted in two distinct environments to cover different use cases:

- **Host (PC):** Windows 11 25H2 (WSL2 6.6.87.2) on an x86_64 AMD Ryzen 9 7900X, running Python 3.13.9 and Scikit-learn 1.6.1 [15].
- **Embedded:** Raspberry Pi Pico utilizing the Pi Pico SDK V2.2.0 [16].

Tsetlin Machine default parameters The default hyperparameters established during the development of our C `Tsetlin` classifier are used as a baseline. They are detailed in Table 7.1. These values were maintained across all Tsetlin Machine models to ensure a fair comparison. However, we observed that the *green_tsetlin* [1] library uses undocumented optimizations, such as the `literal_budget` parameter, which complicates the verification of its underlying model structure.

Metrics The benchmarks on the host were executed via Jupyter Notebooks using our Python bindings. This enabled a direct performance comparison against Scikit-learn models and the *green_tsetlin* [1] library. We tracked the following metrics: Accuracy on Training Dataset, Accuracy on Testing Dataset, Training Time, Prediction Time on Training Dataset, Prediction Time on Testing Dataset, Estimated Model Size. The model size for TM-based models is estimated based on the internal data structures, while Scikit-learn models use Python `pickle` exports. Consequently, both these figures and the run times should be compared with caution due to implementation differences.

Embedded hardware In order to assess the behavior of our implementation, we analyzed how specific hyperparameters, epoch counts, and batch numbers influence model learning. As a final proof of concept, we deployed a minimal Noisy-XOR example on the Raspberry Pi Pico using only the core C library, demonstrating the feasibility of application on resource-constrained hardware.

Table 7.1.: Standardized default hyperparameters across Tsetlin Machine implementations, established using the C Tsetlin classifier baseline.

Model	Default Parameters
C Tsetlin	threshold=1000, num_clauses=1000, max_state=127, min_state=-127, boost_true_positive_feedback=False, s=3.0, epochs=10
C Tsetlin Sparse	threshold=1000, num_clauses=1000, max_state=127, min_state=-127, boost_true_positive_feedback=False, s=3.0, epochs=10
Green Tsetlin	threshold=1000, n_clauses=1000, boost_true_positives=False, s=3.0, n_epochs=10, literal_budget=None
Green Tsetlin Sparse	threshold=1000, n_clauses=1000, boost_true_positives=False, s=3.0, n_epochs=10, literal_budget=None, dynamic_AL=True, lower_ta_threshold=-40, active_literals_size=100, clause_size=50

7.1. Noisy-XOR Dataset

In the evaluation of our implementation, we first utilized the Noisy-XOR problem, a common benchmark for Tsetlin Machines introduced in [5]. This problem is often challenging for many machine learning models because the XOR relation is non-linearly separable and its complexity increases when noise is added.

Dataset and problem representation The dataset consists of 1000 randomly generated samples with 16 features. The true target label was determined by the XOR sum of only the first two features (e.g. $x : [0, 1, 0, \dots, 0], y : 1$), while the remaining 14 features are random noise. To simulate real-world conditions, a noise level of 10% was introduced by flipping the target bit.

Preprocessing and input format The final dataset was divided into a training set (80%) and a testing set (20%), maintaining class proportions of approximately 53% for Class 0 and 47% for Class 1. Because TMs operate using Boolean algebra and the generated data was already binary, no additional preprocessing was required, and the raw bits were passed directly to the models.

Model hyperparameters We aimed to evaluate the base models using the default parameters while tuning specific configurations via `GridSearchCV`. The parameters used are detailed in Table 7.2. For this problem, which features only 16 binary inputs, `GridSearchCV` determined that 64 clauses were sufficient for optimal performance.

Results and comparison The results in Table 7.3 demonstrate that our `C Tsetlin` classifier is highly competitive, matching the accuracy of established machine learning models and performing closely to the `Green Tsetlin` classifier. Due to the 10% added noise, we expect model accuracy to cap at 0.9 on the training dataset unless overfitting occurs (as seen in the `Random Forest` and `MLP` models). On the testing dataset, our implementation tied for the best result alongside `SVC` and `Green Tsetlin` classifier. Slightly longer training and prediction time are likely the result of `tsetlin_machine_c` not implementing vectorization, as decided in the design (see Section 4.2). Furthermore, the version optimized via `GridSearchCV` is significantly smaller and faster than the version using default parameters, highlighting the importance of parameter tuning.

Table 7.2.: Hyperparameters and GridSearchCV search spaces for the Noisy-XOR dataset. The table lists only parameters that differ from model defaults; for GridSearchCV the best-performing configuration is indicated in **bold**.

Model	Modified Parameters
Logistic Regression	max_iter=1000
Decision Tree	--
Random Forest	--
MLP	max_iter=1000, solver="lbfgs"
SVC	--
Linear SVC	dual="auto"
K-Nearest Neighbors	--
Gaussian Naive Bayes	--
Gradient Boosting	--
LightGBM	--
GridSearch Random Forest	n_estimators: [<u>50</u> , 100, 200], max_depth: [<u>None</u> , 10, 20, 30], min_samples_split: [2, 5, <u>10</u>]
GridSearch MLP	hidden_layer_sizes: [(50,), (<u>100</u> ,), (50, 50)], activation: [<u>"relu"</u> , "tanh"], alpha: [0.0001, 0.001, <u>0.01</u>]
Green Tsetlin	--
Green Tsetlin Sparse	--
C Tsetlin	--
GridSearch C Tsetlin	boost_true_positive_feedback: [<u>False</u> , True], num_clauses: [<u>64</u> , 125, 500], s: [<u>3</u> , 9]
C Tsetlin Sparse	--
GridSearch C Tsetlin Sparse	boost_true_positive_feedback: [<u>False</u> , True], num_clauses: [<u>64</u> , 125, 500], s: [<u>3</u> , 9]

Table 7.3.: Benchmark results for the Noisy-XOR dataset (evaluated on the Host (PC) environment). Model sizes are estimated based on internal data structures for Tsetlin-based models and via `pickle` exports for standard models, requiring caution when comparing directly. The best results are indicated in **bold**.

Model	Train Acc.	Test Acc.	Train Time (s)	Train Pred. Time (s)	Test Pred. Time (s)	Model Size (B)
Logistic Regression	0.5487	0.4750	0.0028	0.0005	0.0003	1,122
Decision Tree	0.9962	0.7350	0.0020	0.0004	0.0004	33,536
Random Forest	0.9962	0.8700	0.0790	0.0068	0.0032	4,127,036
MLP	0.9962	0.8150	0.1067	0.0008	0.0004	35,897
SVC	0.8950	0.9300	0.0192	0.0146	0.0040	104,823
Linear SVC	0.5450	0.4750	0.0013	0.0008	0.0004	1,006
K-Nearest Neighbors	0.8250	0.7650	0.0008	0.0217	0.0062	20,179
Gaussian Naive Bayes	0.5513	0.4700	0.0012	0.0009	0.0004	1,373
Gradient Boosting	0.8962	0.9150	0.0767	0.0017	0.0006	139,263
LightGBM	0.9850	0.8950	0.0502	0.0016	0.0007	350,233
GS Random Forest	0.9200	0.9150	2.9232	0.0030	0.0015	622,791
GS MLP	0.9962	0.8500	1.8754	0.0008	0.0003	39,978
Green Tsetlin	0.8925	0.9300	0.2858	0.0069	0.0016	38,008
Green Tsetlin Sparse	0.8925	0.9300	1.7496	0.0031	0.0010	807,608
C Tsetlin	0.8900	0.9300	0.3476	0.0147	0.0039	43,008
GS C Tsetlin	0.8925	0.9300	0.4142	0.0013	0.0006	2,760
C Tsetlin Sparse	0.8450	0.9050	0.4636	0.0141	0.0038	144,352
GS C Tsetlin Sparse	0.8925	0.9300	0.4473	0.0009	0.0005	9,144

7.2. Digits Dataset

Handwritten digit recognition is a standard benchmark task in machine learning. The Digits dataset is included in the Scikit-learn [15] library and is based on the “Pen-Based Recognition of Handwritten Digits” [17].

Dataset and problem representation The dataset consists of 1797 grayscale 8×8 images, examples of which are shown in Figure 7.1. Each pixel has a brightness level ranging from 0 to 16. To utilize these images in our model, we represent them as a flat feature vector of length 64.

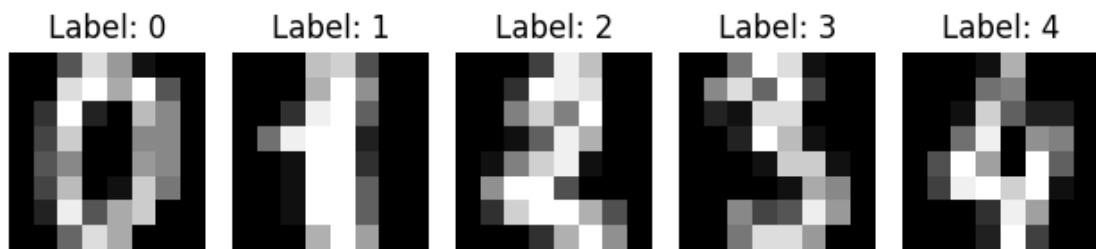


Figure 7.1.: Five 8×8 samples from the Digits dataset with brightness levels from 0 to 16. Correct labels are shown above.

Preprocessing and input format This task involves multiclass classification $y: [0, 1, \dots, 9]$. Because the input vectors contain features with multiple possible values, they must be transformed into a binary format compatible with the Tsetlin Machine. We achieved this using Scikit-learn’s `OneHotEncoder`, which expanded the feature set from $X: (1797, 64)$ with values in $[0, 1, \dots, 16]$ to $X: (1797, 890)$ with binary values $[0, 1]$. The data was divided using a typical 80% training and 20% testing split while maintaining class proportions.

Model hyperparameters Similar to the Noisy-XOR evaluation, we maintained default parameters while only modifying the `GridSearchCV` space for our classifier, as shown in Table 7.4. Given the significantly higher number of features in this dataset, the parameter search determined that increasing the number of clauses to 1500 was beneficial for performance.

Results and comparison The results summarized in Table 7.5 demonstrate that Tsetlin Machine models can handle more complex datasets. However, because the input format is restricted to binary, the models require a higher number of clauses to capture relevant patterns. This requirement leads to an increase in both time and memory consumption compared to other models. These findings suggest that the binary preprocessing required

for TMs may not be ideal for high-dimensional image recognition tasks. Regardless, our implementation maintained accuracy levels comparable to the *green_tsetlin* [1] library, likely confirming the correctness of our implementation. Slightly longer training and prediction time are likely the result of *tsetlin_machine_c* not implementing vectorization, as decided in the design (see Section 4.2).

Table 7.4.: Hyperparameters and `GridSearchCV` search spaces for the Digits dataset. The table lists only parameters that differ from model defaults; for `GridSearchCV` the best-performing configuration is indicated in **bold**.

Model	Modified Parameters
Logistic Regression	<code>max_iter=1000</code>
Decision Tree	--
Random Forest	--
MLP	<code>max_iter=1000, solver="lbfgs"</code>
SVC	--
Linear SVC	<code>dual="auto"</code>
K-Nearest Neighbors	--
Gaussian Naive Bayes	--
Gradient Boosting	--
LightGBM	--
GridSearch Random Forest	<code>n_estimators: [50, 100, 200], max_depth: [None, 10, 20, 30], min_samples_split: [2, 5, 10]</code>
GridSearch MLP	<code>hidden_layer_sizes: [(50,), (100,), (50, 50)], activation: ["relu", "tanh"], alpha: [0.0001, 0.001, 0.01]</code>
Green Tsetlin	--
Green Tsetlin Sparse	--
C Tsetlin	--
GridSearch C Tsetlin	<code>boost_true_positive_feedback: [False, True], num_clauses: [500, 1000, 1500], s: [3, 9]</code>
C Tsetlin Sparse	--
GridSearch C Tsetlin Sparse	<code>boost_true_positive_feedback: [False, True], num_clauses: [64, 125, 500, 1000], s: [3, 9]</code>

Table 7.5.: Benchmark results for the Digits dataset (evaluated on the Host (PC) environment). Model sizes are estimated based on internal data structures for Tsetlin-based models and via `pickle` exports for standard models, requiring caution when comparing directly. The best results are indicated in **bold**.

Model	Train Acc.	Test Acc.	Train Time (s)	Train Pred. Time (s)	Test Pred. Time (s)	Model Size (B)
Logistic Regression	<u>1.0000</u>	0.9528	1.5764	<u>0.0035</u>	0.0042	84,514
Decision Tree	<u>1.0000</u>	0.7833	0.0472	0.0038	<u>0.0024</u>	<u>59,928</u>
Random Forest	<u>1.0000</u>	0.9444	0.2000	0.0159	0.0068	9,855,780
MLP	<u>1.0000</u>	0.9306	4.5576	0.0226	0.0338	1,461,199
SVC	0.9965	<u>0.9583</u>	0.2974	0.3944	0.1206	9,585,426
Linear SVC	<u>1.0000</u>	0.9472	0.1006	0.0137	0.0068	84,395
K-Nearest Neighbors	0.9088	0.8667	<u>0.0028</u>	0.0679	0.0101	10,256,123
Gaussian Naive Bayes	0.9666	0.8278	0.0062	0.0219	0.0052	155,647
Gradient Boosting	<u>1.0000</u>	0.9139	9.5533	0.0150	0.0053	1,359,941
LightGBM	<u>1.0000</u>	0.9472	0.6413	0.0092	0.0031	3,124,964
GS Random Forest	<u>1.0000</u>	0.9444	5.6181	0.0347	0.0139	19,661,484
GS MLP	<u>1.0000</u>	0.9361	8.0691	0.0346	0.0080	1,465,209
Green Tsetlin	0.9923	0.9222	1.8070	0.0181	0.0063	1,794,040
Green Tsetlin Sparse	0.9402	0.8889	16.0517	0.0113	0.0042	822,040
C Tsetlin	0.9882	0.9222	18.6141	0.7369	0.1846	1,831,040
GS C Tsetlin	0.9937	0.9139	121.3198	1.0595	0.2674	2,746,540
C Tsetlin Sparse	0.6715	0.5972	21.0368	0.3977	0.1014	970,992
GS C Tsetlin Sparse	0.8045	0.7083	49.0949	0.3634	0.0939	967,712

7.3. Fashion-MNIST Dataset

The Fashion-MNIST dataset [18] is a popular image classification task developed as a drop-in replacement for the original MNIST dataset. We utilized this dataset to test the performance limits and scaling capabilities of the Tsetlin Machine, as it is more difficult compared to the Digits dataset. This evaluation is conducted primarily for research purposes as we do not expect our implementation to be used in this types of task otherwise.

Dataset and problem representation The dataset consists of 70000 grayscale images of Zalando’s clothing articles. Each example is a 28×28 pixel image (brightness levels 0-255) associated with one of the 10 classes. Sample images from the dataset are illustrated in Figure 7.2.

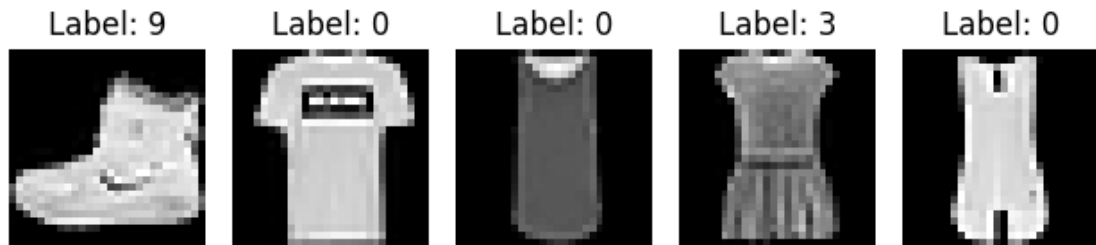


Figure 7.2.: Five 28×28 samples from the Fashion-MNIST dataset with brightness levels from 0 to 255 before preprocessing. Correct labels are shown above.

Preprocessing and input format To prepare the data for the TM, we selected a subset consisting of the first 10000 samples. The pixel values were normalized using the division by 16 and cast to unsigned 8-bit integers. The original resolution was down-sampled to 14×14 by selecting every second pixel in both dimensions. These images were then flattened into vectors of 196 features. Since TMs require binary input, these features were transformed using Scikit-learn’s `OneHotEncoder` with a maximum of 8 categories. This process transformed the input dimensionality from $X: (10000, 784)$ with values in $[0, 1, \dots, 255]$ to $X: (10000, 1555)$ binary features. Finally, the data was divided into 80% training and 20% testing sets while maintaining the class proportions.

Model hyperparameters Due to the significant increase in training times with this dataset, we restricted the baseline models only to the Support Vector Classifier (SVC), which is a common method in image recognition tasks. As shown in Table 7.6, all the models evaluated were kept at their default parameter values.

Table 7.6.: Hyperparameters for the Fashion-MNIST dataset. The table lists only parameters that differ from model defaults.

Model	Modified Parameters
SVC	--
Green Tsetlin	--
Green Tsetlin Sparse	--
C Tsetlin	--
C Tsetlin Sparse	--

Results and comparison The results in Table 7.7 indicate that this dataset contains too many features for the current binary approach to TMs to remain efficient. This reveals a limitation in scaling, as all processing times increase significantly. Although both training and testing accuracy were lower than those of the SVC, prediction times for TM-based models remained faster. In particular, our implementation took significantly longer processing times than the *green_tsetlin* [1] library across all metrics. These findings suggest that the increase in dimensionality and the number of training samples creates performance bottlenecks that are more pronounced in our current implementation than in existing third-party libraries, likely because *tsetlin_machine_c* does not implement vectorization, as decided in the design (see Section 4.2).

Table 7.7.: Benchmark results for the Fashion-MNIST dataset (evaluated on the Host (PC) environment). Model sizes are estimated based on internal data structures for Tsetlin-based models and via `pickle` exports for standard models, requiring caution when comparing directly. The best results are indicated in **bold**.

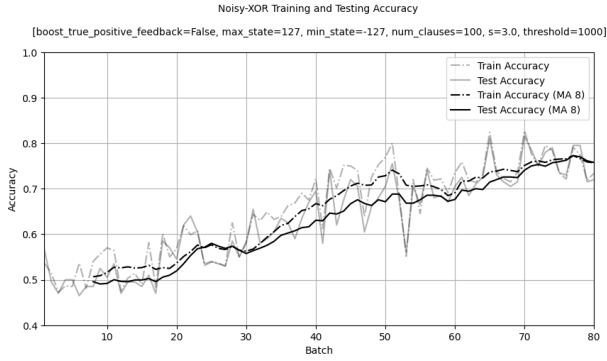
Model	Train Acc.	Test Acc.	Train Time (s)	Train Pred. Time (s)	Test Pred. Time (s)	Model Size (B)
SVC	0.9329	0.8260	8.3000	32.5356	8.2058	66,885,009
Green Tsetlin	0.8053	0.7785	15.9906	0.0475	0.0136	3,124,040
Green Tsetlin Sparse	0.6094	0.5965	201.6389	0.0518	0.0151	822,040
C Tsetlin	0.7600	0.7320	209.8398	6.5704	1.6151	3,161,040
C Tsetlin Sparse	0.5821	0.5810	633.4991	3.1626	0.7933	1,271,776

7.4. Incremental Noisy-XOR Learning

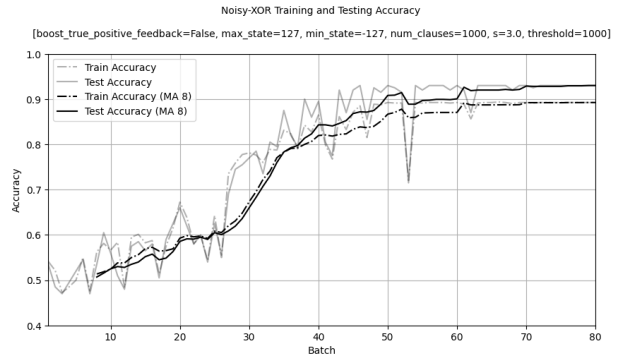
The performance of Tsetlin Machines is highly sensitive to hyperparameter selection, which is often counter-intuitive. This marks the importance of performing an optimal configuration search using tools such as Grid Search. In this section, we evaluate the learning behavior of our model across sequential data batches to understand how specific impact incremental learning.

The training and testing accuracy curves per batch are illustrated in Figure 7.3. A comparison between models with different clause counts demonstrated that using 1000 clauses allows the model to learn faster and achieve higher stability compared to using only 100 clauses. This behavior shown in Figures 7.3a and 7.3b suggests that a larger pool of clauses may allow individual units to “catch up” to patterns more quickly.

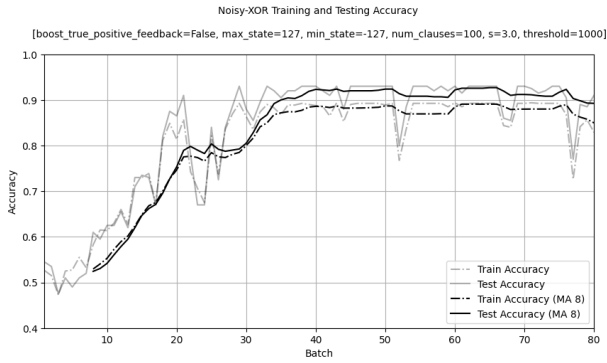
The experimental results presented in Figures 7.3b and 7.3d indicate that while increasing the number of epochs per batch improves accuracy on the current batch, it leads to the degradation of previously learned patterns, resulting in worse overall performance. This drop in accuracy is likely a consequence of the TM’s internal forgetting mechanism during feedback processing. This presents a challenge for incremental learning, where maintaining a balance between retaining knowledge and memorizing new patterns is crucial.



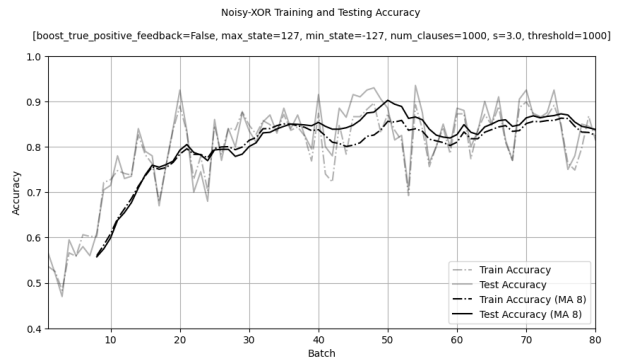
(a) 100 clauses, 1 epoch per batch



(b) 1000 clauses, 1 epoch per batch



(c) 100 clauses, 10 epochs per batch



(d) 1000 clauses, 10 epochs per batch

Figure 7.3.: Train accuracy (dash-dot line) and test accuracy (solid line) per batch for the Noisy-XOR dataset. The evaluation tracks progress over 80 batches of 10 samples each.

7.5. Embedded Noisy-XOR Benchmarks

We demonstrated the capability of our implementation by running the minimal version of the core C library on resource-constrained hardware. As a target platform, we chose the Raspberry Pi Pico development board due to its popularity and software support, specifically the Pico-SDK [16]. The platform is powered by the RP2040 microcontroller chip, featuring a dual-core Arm Cortex-M0+ processor clocked at 133 MHz, 264 KB of SRAM and 2 MB of on-board flash memory.

We conducted benchmarks on the Noisy-XOR dataset similar to those performed in Python but using a wider range of configurations. Selected tests were performed on a desktop PC running Ubuntu OS in cases where the models exceeded the hardware performance limits of the Raspberry Pi Pico or where peak memory statistics were unavailable. The results are detailed in Table 7.8, with values gathered from the Ubuntu environment marked with a (U) prefix.

The analysis reveals several important characteristics:

- Tsetlin Machine memory usage (**TMM**) is directly proportional only to the number of clauses.
- Training time (**TRT**) is proportional to the number of clauses, samples, and epochs.
- Evaluation time (**EVT**) is also proportional to the number of clauses, and samples.
- Other hyperparameters than mentioned above have a relatively small impact on performance when set to values close to established model defaults.
- Accurately measuring peak memory usage (**PMU**) proved difficult in the Ubuntu environment due to system overhead.

As shown in Table 7.8, our implementation remains highly usable on embedded hardware. For example, configurations **9** and **10** achieve accuracy levels close to the theoretical maximum (counting for the 10% added noise). We can observe a clear trade-off: using more epochs and fewer clauses (Configuration **10**) results in a longer training time but a smaller memory footprint and faster prediction speeds, which is ideal for pretrained models. In contrast, using a single epoch with more clauses (Configuration **9**) allows faster on-device learning at the cost of increased model size and slower evaluation.

Table 7.8.: Noisy-XOR results on Raspberry Pi Pico and Ubuntu Desktop (the Host (PC) environment), testing how hyperparameters affect accuracy and execution time. Peak RAM usage was calculated on the Ubuntu Desktop and includes system overhead. Due to hardware performance limits, configurations exceeding the 1000 clauses were benchmarked only on Ubuntu (marked with (U) prefix).

N	SAM	CLS	THR	LIT	CLU	MXS	MNS	BST	SNS	EPC	TRT	EVT	COP	TOP	ACC	TMM	DSM	PMU
1	1000	2	1000	16	1000	127	-127	0	3.0	1	24.73	0.88	183	200	0.92	36.21	19.53	(U)1728.0
2	1000	2	1000	16	1000	127	-127	0	3.0	2	41.30	0.87	183	200	0.92	36.21	19.53	(U)1728.0
3	1000	2	1000	16	1000	127	-127	0	3.0	4	72.51	0.88	183	200	0.92	36.21	19.53	(U)1728.0
4	1000	2	1000	16	1000	63	-63	0	3.0	1	24.73	0.88	183	200	0.92	36.21	19.53	(U)1728.0
5	1000	2	1000	16	1000	31	-31	0	3.0	1	24.73	0.88	183	200	0.92	36.21	19.53	(U)1728.0
6	1000	2	1000	16	1000	15	-15	0	3.0	1	24.51	0.86	183	200	0.92	36.21	19.53	(U)1728.0
7	1000	2	1000	16	500	127	-127	0	3.0	1	12.92	0.40	183	200	0.92	18.15	19.53	(U)1728.0
8	1000	2	1000	16	250	127	-127	0	3.0	1	6.79	0.21	183	200	0.92	9.12	19.53	(U)1728.0
9	1000	2	1000	16	125	127	-127	0	3.0	1	3.54	0.10	183	200	0.92	4.6	19.53	(U)1728.0
10	1000	2	1000	16	64	127	-127	0	3.0	10	11.93	0.05	183	200	0.92	2.39	19.53	(U)1728.0
11	1000	2	1000	16	1000	127	-127	0	3.0	10	163.19	0.89	182	200	0.91	36.21	19.53	(U)1728.0
12	1000	2	500	16	1000	127	-127	0	3.0	1	22.98	0.87	182	200	0.91	36.21	19.53	(U)1728.0
13	1000	2	1000	16	1000	127	-127	0	3.0	1	24.53	0.82	181	200	0.91	36.21	19.53	(U)1728.0
14	1000	2	250	16	1000	127	-127	0	3.0	1	22.58	0.89	180	200	0.9	36.21	19.53	(U)1728.0
15	1000	2	1000	16	1000	127	-127	0	6.0	1	26.33	0.67	179	200	0.9	36.21	19.53	(U)1728.0
16	1000	2	32	16	1000	127	-127	0	3.0	1	21.30	0.87	179	200	0.9	36.21	19.53	(U)1728.0
17	250	2	1000	16	1000	127	-127	0	3.0	10	47.14	0.22	44	50	0.88	36.21	4.88	(U)1728.0
18	1000	2	125	16	1000	127	-127	0	3.0	1	21.54	0.88	173	200	0.87	36.21	19.53	(U)1728.0
19	1000	2	64	16	1000	127	-127	0	3.0	1	21.03	0.88	170	200	0.85	36.21	19.53	(U)1728.0
20	1000	2	32	16	500	127	-127	0	3.0	1	10.87	0.44	168	200	0.84	18.15	19.53	(U)1728.0
21	1000	2	32	16	500	127	-127	0	3.0	10	79.82	0.46	166	200	0.83	18.15	19.53	(U)1728.0
22	1000	2	1000	16	1000	7	-7	0	3.0	1	27.17	0.81	165	200	0.83	36.21	19.53	(U)1728.0
23	1000	2	1000	16	64	127	-127	0	3.0	1	1.85	0.05	162	200	0.81	2.39	19.53	(U)1728.0
24	1000	2	16	16	1000	127	-127	0	3.0	1	21.27	0.85	161	200	0.81	36.21	19.53	(U)1728.0
25	500	2	1000	16	1000	127	-127	0	3.0	1	36.21	0.44	80	100	0.8	36.21	9.77	(U)1728.0
26	1000	2	32	16	500	127	-127	1	3.0	1	10.74	0.42	150	200	0.75	18.15	19.53	(U)1728.0
27	1000	2	1000	16	64	127	-127	1	3.0	1	1.79	0.05	142	200	0.71	2.39	19.53	(U)1728.0
28	1000	2	1000	16	1000	127	-127	0	1.5	1	30.84	1.15	124	200	0.62	36.21	19.53	(U)1728.0
29	250	2	1000	16	1000	127	-127	0	3.0	1	7.21	0.22	27	50	0.54	36.21	4.88	(U)1728.0
30	1000	2	1000	16	1000	3	-3	0	3.0	1	28.43	0.73	100	200	0.5	36.21	19.53	(U)1728.0
31	1000	2	1000	16	10000	127	-127	0	3.0	1	(U)1.16	(U)0.11	172	200	0.86	361.45	19.53	(U)2112.0
32	1000	2	1000	16	100000	127	-127	0	3.0	1	(U)11.61	(U)1.17	167	200	0.84	3613.4	19.53	(U)5376.0
33	1000	2	1000	16	200000	127	-127	0	3.0	1	(U)23.00	(U)2.32	165	200	0.83	7226.68	19.53	(U)8832.0

Legend: N: Configuration Number, **SAM**: Number of Samples, **CLS**: Number of Classes, **THR**: Threshold, **LIT**: Number of Literals, **CLU**: Number of Clauses, **MXS**: Maximum State, **MNS**: Minimum State, **BST**: Boost True Positive Feedback, **SNS**: Learning Sensitivity (s), **EPC**: Training Epochs, **TRT**: Training Time [s], **EVT**: Evaluation Time [s], **COP**: Correct Predictions, **TOP**: Total Predictions, **ACC**: Accuracy, **TMM**: TM Memory Usage [KB], **DSM**: Dataset Memory Usage (static) [KB], **PMU**: Peak Memory Usage on Ubuntu Desktop [KB]

7.6. Summary

The initial evaluation of our Tsetlin Machine implementations shows potential, especially for resource-limited environments dealing with not overly high-dimensional data. We observed the impact of hyperparameters, mainly number of clauses and training epochs, making it essential to tune them for each task. We observed the mechanism inside TM designed to prevent overfitting, creating a challenge in incremental learning, where the balance between retaining old knowledge and memorizing new patterns is crucial. Furthermore, benchmarking was complicated due to the *green_tsetlin* [1] library design, which includes some undocumented optimizations (e.g. `literal_budget` parameter that is not properly described).

8. Conclusions

The previous chapters have described the goals of the project. This concluding chapter reflects on the objectives that were achieved successfully and identifies any areas of improvement. It also proposes further work, research, and improvements to the project as well as TMs in general.

8.1. Achieved Results

The project work resulted in a functional, stable and optimized open-source product, pending only cosmetic or experiential refinements. All base requirements were fulfilled – the Tsetlin Machine execution engine can be used to both infer and train in a limited resource environment, model data is stored in an efficient, easy to utilize format. The project further mitigates a gap in the ecosystem encountered during its development – the lack of available knowledge sources and documentation on how to put the TM theory into practice. The product is suitable for production use, though compilation for non-standard or specialized architectures requires additional effort.

8.2. Further Work

During the development period of the project, numerous promising ideas emerged that warrant further investigation. These ideas merit further investigation and could meaningfully improve both the research outcomes and the overall robustness of the project.

8.2.1. Cross Compilation

An immediate extension that would increase the utility and reach of this project is the introduction of a formal cross-compilation pipeline. Implementing cross-compilation would enable automated builds for multiple target architectures (e.g. ARM, RISC-V, x86_64) and ABIs, improving portability and facilitating deployment on embedded and heterogeneous systems. Addressing this would broaden its applicable use cases.

8.2.2. Coalesced TM and the Linear Layer

Seeing the similarities between TM and a Perceptron [11], further parallels could be drawn between a Coalesced TM and a whole linear layer with an activation function, like in a Neural Network. More research is necessary to find out whether they could be used interchangeably. The results Likely would vary greatly based on the complexity of the task and the data structure. Another idea to explore is whether TMs could be combined sequentially, creating a larger model, with or without other layer types.

Due to how customizable *tsetlin_machine_c* is, a custom feedback function and an output activation function can be written to facilitate a TM to be used as a layer of a larger model – painlessly, without the need to rewrite the main logic of the execution engine.

8.2.3. Integer Only Implementation of TM

Some processors and compute units do not natively provide a floating-point unit, especially on embedded systems and similar hardware. TM design typically requires floating point operations, however authors believe it should not be difficult to implement.

All floating point operations in TM design originate from the parameter s – learning sensitivity – which handles how often specific feedback types should apply. This is traditionally done using floating point. However, rewriting this behavior to integer-only logic seems rather straightforward.

Bibliography

- [1] Sondre Glimsdal et al. *Green Tsetlin Redefining Efficiency in Tsetlin Machine Frameworks*. 2024. arXiv: [2405.04212](https://arxiv.org/abs/2405.04212) [cs.AI]. URL: <https://arxiv.org/abs/2405.04212>.
- [2] Ole-Christoffer Granmo. *TsetlinMachine*. <https://github.com/cair/TsetlinMachine>. 2018.
- [3] Ole-Christoffer Granmo. *TsetlinMachineC*. <https://github.com/cair/TsetlinMachineC>. 2018.
- [4] Ole-Christoffer Granmo. *PyTsetlinMachineCUDA*. <https://github.com/cair/PyTsetlinMachineCUDA>. 2020.
- [5] Ole-Christoffer Granmo. *The Tsetlin Machine – A Game Theoretic Bandit Driven Approach to Optimal Pattern Recognition with Propositional Logic*. 2021. arXiv: [1804.01508](https://arxiv.org/abs/1804.01508) [cs.AI]. URL: <https://arxiv.org/abs/1804.01508>.
- [6] Sondre Glimsdal and Ole-Christoffer Granmo. *Coalesced Multi-Output Tsetlin Machines with Clause Sharing*. 2021. arXiv: [2108.07594](https://arxiv.org/abs/2108.07594) [cs.AI]. URL: <https://arxiv.org/abs/2108.07594>.
- [7] Ole-Christoffer Granmo. *An Introduction to Tsetlin Machines*. Chapter 1: Your First Tsetlin Machine (Revised 7/11/22). 2022. URL: https://tsetlinmachine.org/wp-content/uploads/2022/11/Tsetlin_Machine_Book_Chapter_One_Revised.pdf.
- [8] Ole-Christoffer Granmo. *An Introduction to Tsetlin Machines*. Chapter 2: Classification (13/11/22). 2022. URL: https://tsetlinmachine.org/wp-content/uploads/2022/11/Tsetlin_Machine_Book_Chapter_2.pdf.
- [9] Michael Lvovitch Tsetlin. “On behaviour of finite automata in random medium”. In: *Avtomat. i Telemekh* 22.10 (1961), pp. 1345–1354.
- [10] Kumpati S Narendra and Mandayam AL Thathachar. *Learning automata: an introduction*. Courier corporation, 2012.
- [11] Jivitesh Sharma, Ole-Christoffer Granmo, and Lei Jiao. *On the Equivalence of the Weighted Tsetlin Machine and the Perceptron*. 2022. arXiv: [2212.13634](https://arxiv.org/abs/2212.13634) [cs.LG]. URL: <https://arxiv.org/abs/2212.13634>.
- [12] Sebastian Østby, Tobias M. Brambo, and Sondre Glimsdal. *The Sparse Tsetlin Machine: Sparse Representation with Active Literals*. 2024. arXiv: [2405.02375](https://arxiv.org/abs/2405.02375) [cs.LG]. URL: <https://arxiv.org/abs/2405.02375>.

- [13] Wouter van Oortmerssen and Google. *FlatBuffers: Memory Efficient Serialization Library*. <https://github.com/google/flatbuffers>. Accessed: 2025-11-13.
- [14] Igor Sikora and Robert Meseke. *Tsetlin Machines execution engine*. https://github.com/notEloiir/tsetlin_machine_c. Accessed: 2026-01-02.
- [15] F. Pedregosa et al. “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.
- [16] Raspberry Pi. *The Raspberry Pi Pico SDK*. <https://github.com/raspberrypi/pico-sdk>. Accessed: 2025-11-13.
- [17] E. Alpaydin and Fevzi. Alimoglu. *Pen-Based Recognition of Handwritten Digits*. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5MG6K>. 1996.
- [18] Han Xiao, Kashif Rasul, and Roland Vollgraf. *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*. Aug. 28, 2017. arXiv: [cs.LG/1708.07747](https://arxiv.org/abs/1708.07747) [[cs.LG](https://arxiv.org/abs/1708.07747)].